

FastFuel

Projektdokumentáció

Császár Zoltán
Massman Noah James
Tircsi Kolos Mátyás

Verzió: 1.0
Dátum: [2026/04/28]

1 Tartalomjegyzék

FastFuel.....	1
1 Tartalomjegyzék	2
2 Bevezetés	6
2.1 A projekt leírása	6
2.2 Célközönség	6
2.3 A projekt hatóköre	6
2.4 Online demo	6
3 Specifikáció	7
3.1 Technológiák és függőségek	7
3.2 Használt szoftverek	7
3.3 Funkcionális követelmények	7
3.4 Nem funkcionális követelmények	8
4 Fejlesztési módszertanok	9
4.1 Csapatszerepek és felelősségek	9
5 Használati útmutató	10
5.1 Üdvözlőoldal	10
5.2 Bejelentkezés és regisztráció	10
5.2.1 Regisztráció	10
5.2.2 Bejelentkezés	10
5.3 Főmenü	11
5.4 Menedzsment oldalak	11
5.4.1 Az oldal áttekintése	11
5.4.2 Rekord felvétele	12
5.4.3 Rekord szerkesztése	12
5.4.4 Rekord törlése	13
5.5 A rendelés folyamata	13
5.5.1 Étteremválasztó ablak	13
5.5.2 Étlap böngészése	14
5.5.3 Termék hozzáadása	14
5.5.4 Rendelési előzmények	16
5.5.5 Rendelés státusz oldal	17
5.5.6 Egy állomás elvégzendő feladatai	17
5.6 Profil oldal	18
5.7 Műszakok	18
5.8 Jogosultságkezelés	19
6 A backend technikai felépítése	21
6.1 Features mappa	21

6.1.1	Features/ <i>feature</i> mappa.....	21
6.1.1.1	Entities	21
6.1.1.2	Configs.....	21
6.1.1.3	DTOs.....	21
6.1.1.4	Mappers.....	21
6.1.1.5	Controllers.....	21
6.1.1.6	Common.....	21
6.1.2	Features/Common mappa	21
6.1.2.1	Interfaces.....	22
6.1.2.2	Services	22
6.1.2.3	Controllers.....	22
6.1.2.4	Permissions	22
6.1.2.5	DbContexts	22
6.1.2.6	Exceptions.....	22
6.1.3	Features/Auth mappa	22
6.1.4	Features/Permissions mappa.....	23
6.1.5	Features/Pages mappa.....	23
6.1.6	Features/Roles mappa	23
6.1.7	Featues/Users mappa	23
6.1.8	Features/Orders mappa	24
6.2	NSwag OperationProcessor-ok.....	24
6.2.1	CrudOperationSummary	24
6.2.2	MarkAsRequiredIfNonNullable	25
6.2.3	PermissionSchema	25
6.2.4	SwaggerQueryParam	25
6.2.5	UnregisteredStatusCodeResultOperation	25
6.3	Környezeti változók.....	26
6.4	Program.cs	26
7	A frontend technikai felépítése.....	27
7.1	Általános, újrahasznosítható komponensek.....	27
7.1.1	NumericCombobox.....	27
7.1.2	Paper	27
7.1.3	PagePermissionGuard.....	27
7.1.4	SearchablePickerModals.....	27
7.2	EntityManager.....	27
7.2.1	EntityTable.....	27
7.2.2	EntityEditor.....	28
7.3	Layout	29

7.3.1	Header	30
7.3.2	Footer	30
7.4	Főmenü	30
7.5	Regisztráció.....	30
7.6	Bejelentkezés	31
7.7	Kódgenerálás.....	31
7.7.1	Kódgenerálás az OpenAPI séma alapján	31
7.7.2	Kódgenerálás a jogosultságokhoz.....	31
7.8	Middleware-ek	34
7.8.1	AuthenticationMiddleware	34
7.8.2	ErrorResponseMiddleware	34
7.9	Pages mappa.....	34
7.10	Router.....	34
7.11	Hibakezelés	34
7.12	Töltőképnyő	35
7.13	Backend url megadása	35
7.14	Összefoglaló.....	35
8	Adatbázis	37
8.1	ASP.NET Identity táblák	37
8.2	Étlaphoz kapcsolódó táblák	37
8.3	Rendelésekhez kapcsolódó táblák.....	37
8.4	Éttermekhez kapcsolódó táblák	37
9	API dokumentáció	39
9.1	Példák.....	39
9.1.1	GET /api/Allergy	39
9.1.2	POST /api/Auth/login	39
9.1.3	GET /api/User/me	40
9.2	Hitelesítés (Authentication)	40
9.3	Hibakezelés	40
10	Üzembehelyezési útmutató	41
10.1	Éles környezet.....	41
10.1.1	Előfeltételek	41
10.1.2	Konfiguráció	41
10.1.3	Beüzemelés	41
10.2	Demo környezet.....	41
10.3	Fejlesztési környezet.....	42
10.3.1	Előfeltételek	42
10.3.1.1	Backend.....	42

10.3.1.2	Frontend	42
10.3.2	Környezeti változók	42
11	Tesztelés.....	43
11.1	Áttekintése	43
11.2	Egységtesztek.....	43
11.3	API tesztek	43
11.4	E2E manuális tesztek	43
12	Jövőbeli fejlődési lehetőségek	44
12.1	Funkcionális lehetőségek	44
12.2	Nem funkcionális lehetőségek	44
13	Felhasznált források	45

2 Bevezetés

2.1 A projekt leírása

A FastFuel projekt célja, hogy egy étteremláncnak egy egyszerűen használható, átlátható és jól skálázható rendszert nyújtson.

2.2 Célközönség

A projekt egy 5-10 étteremből álló étteremláncnak lett szánva, ami szeretné a teljes munkafolyamat központosítani.

2.3 A projekt hatóköre

A rendszer magában foglalja:

- az étlap kezelését, beleértve az ételeket, menüket, összetevőket és allergéneket,
- a rendelések felvételét dolgozói felületen és önkiszolgáló kioszkról egyaránt,
- a konyhai feladatok automatikus szétosztását az érintett állomások között,
- a dolgozók konyhai állomásokra való betanításának nyilvántartását,
- a rendelési állapot valós idejű megjelenítését a lobbyban várakozó vendégek számára,
- a műszakbeosztás kezelését és a dolgozók saját beosztásának megtekintését,
- Az egyes éttermekhez tartozó adatok (helyszín, elérhetőség, nyitvatartás) nyilvántartását,
- a felhasználók regisztrációját és kezelését különböző típusok szerint (vásárló, dolgozó, admin, gép),
- tetszőleges számú szerepkör létrehozását és jogosultságok részletes szabályozását.

A rendszer nem terjed ki:

- készletgazdálkodásra és alapanyag-rendelésre,
- pénzügyi vagy számlázási funkciókra,
- külső futárszolgáltatások kezelésére,
- marketing- vagy hűségprogramok kezelésére

2.4 Online demo

A projekt Demo üzemmódban elérhető és kipróbálható a <https://fastfuel.cs-z.hu/> linken.

A Demo üzemmódról és a belépési adatokról a [Demo környezet](#) című fejezetben olvashatunk bővebben.

3 Specifikáció

3.1 Technológiák és függőségek

Adatbázis: MariaDB

Backend: .NET10, C#, Entity Framework Core, Pomelo.EntityFrameworkCore.MySql, ASP.NET Core, ASP.NET Core Identity, OpenAPI, Swagger UI

Frontend: React, TypeScript, Mantine UI, Tanstack Query

Production környezet: Docker, Traefik, Oracle Cloud

3.2 Használt szoftverek

IDE: VS Code, Visual Studio, Rider, Webstorm

Adatbázistervezés: DbSchema

Dokumentáció: Word, Draw.io, RapiPDF

Csapatmunka: Git, GitHub, Jira, Discord

Tesztelés: Postman, Burp Suite

3.3 Funkcionális követelmények

Az [1.3](#) pontban tárgyalt funkciók lehetővé tétele. Részletesebben:

1. Étlapkezelés
 - A rendszer lehetővé teszi ételek, menük, összetevők és allergének létrehozását, módosítását és törlését.
 - Minden ételhez meg lehet adni, hogy mely összetevőket tartalmazza, és azok melyik konyhai állomáson készülnek.
 - Az allergének az ételekhez rendelhetők, és az étlapon megjeleníthetők.
2. Rendeléskezelés
 - A vendégek önkiszolgáló kioszkról, a dolgozók belső felületen tudnak rendelést felvenni.
 - A rendszer nyilvántartja a rendelések tartalmát, státuszát és időpontját.
 - A felhasználók megtekinthetik rendelési előzményeiket, melyeknek tartalma akkor sem változik, ha módosul az étlap
3. Konyhai feladatkezelés
 - Egy rendelés beérkezésekor a rendszer automatikusan szétosztja a feladatokat a megfelelő konyhai állomások között, az ételek összetevői alapján.
 - Minden állomás csak a saját feladatait látja.
4. Rendelés állapotának visszajelzése
 - A lobbyban elhelyezett kijelzőn a várakozó vendégek nyomon követhetik rendelésük állapotát (pl. elkészítés alatt, átvehető).
5. Dolgozók betanításának nyilvántartása
 - A rendszer rögzíti, hogy egyes dolgozók mely konyhai állomásokra vannak betanítva.
 - Ez az adat lekérdezhető és módosítható az adminisztrátori felületen.
6. Műszakbeosztás
 - Az adminisztrátorok műszakbeosztást tudnak összeállítani és kezelni.
 - A dolgozók megtekinthetik saját beosztásukat.
7. Éttermi adatok nyilvántartása
 - A rendszerben minden egyes étteremhez rögzíthető a helyszín, elérhetőség és nyitvatartás.
 - Ezek az adatok módosíthatók és lekérdezhetők, valamint láthatóak a főoldalon
8. Felhasználókezelés

- A rendszer támogatja a különböző típusú felhasználók (vásárló, dolgozó, admin, gép/kioszkfiók) regisztrációját és kezelését.
 - Az adminisztrátorok tetszőleges számú szerepkört hozhatnak létre.
 - Egyes szerepkörök automatikusan kioszthatók a felhasználóknak (pl. regisztrációkor).
9. Jogosultságkezelés
- A szerepkörök alapján szabályozható, hogy egyes felhasználók mely menüpontokat látják, és milyen műveleteket hajthatnak végre.
 - A jogosulatlan funkciók nem jelennek meg, vagy le vannak tiltva a felületen.

3.4 Nem funkcionális követelmények

1. Teljesítmény
 - A felület kompakt és elérhető kialakítása megengedi a gyors munkatempót.
2. Biztonság
 - A rendszerben a legtöbb végpont védett, ezek eléréséhez megfelelő jogosultság szükséges.
 - A biztonságos autentikációt a hashelt jelszó és a http-only cookie garantálja.
3. Tesztelhetőség
 - A projekt komponenseit egymástól független könnyen lehet tesztelni.
 - Automata és manuális tesztek is készültek, amik a projekt nagyobb részét lefedik.
4. Felhasználóbarát kialakítás
 - A webkliens mobilbarát.
 - Az oldalakat a lehetőségekhez képest egyszerűen vannak kialakítva

4 Fejlesztési módszertanok

A projekt elkészítéséhez agilis módszereket alkalmaztunk. A feladatok státuszát kanban board-on (Jira) követtük, a munkát kezdetben 2, majd 1 hetes sprintekre bontottuk, egymás munkáját mindig átnéztük, valamint a sprint végén áttekintettük az elkészült és az előttünk álló feladatokat, melyeket Jira-ban tartottunk nyilván. A kommunikáció Discordon és GitHub-on történt, a verziókezeléshez Git-et és GitHub-ot használtunk.

4.1 Csapatszerepek és felelőségek

Császár Zoltán: adatbázis, backend, frontend, dokumentáció, deployment

Tircsi Kolos Mátyás: UI design, backend, frontend, dokumentáció

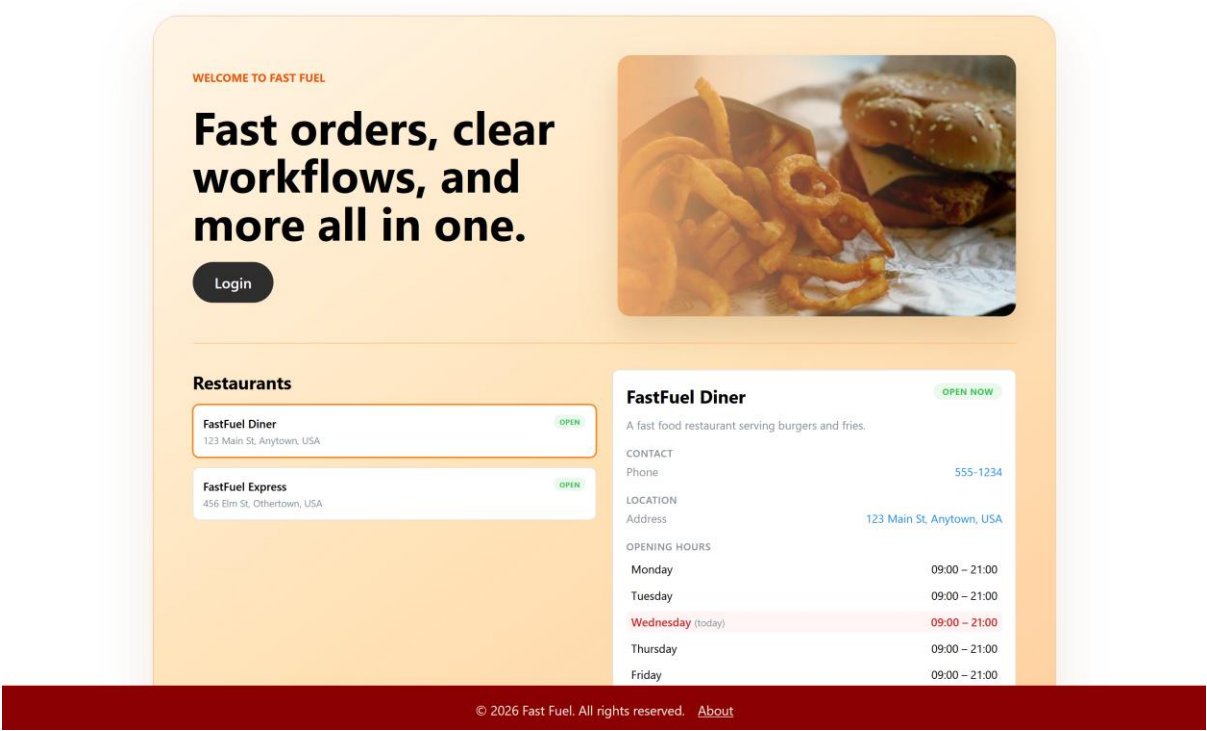
Massman Noah James: backend, frontend, tesztelés, dokumentáció

5 Használati útmutató

5.1 Üdvözlőoldal

Ez az oldal fogad minket először. Megtekinthetők az éttermek nyitvatartásai és elérhetőségei, valamint lehetőséget kínál bejelentkezésre.

Welcome



5.2 Bejelentkezés és regisztráció

A regisztráció és a bejelentkezés egyszerű, magától értetődő. A jelszó szigorúan validálva van és érthetően jelzi a felhasználó fele, ha probléma akadt.

5.2.1 Regisztráció

Welcome

Registration

Login

Name *

Enter your name

Email *

Enter your email

Username *

Enter username

Password *

Enter password

Confirm Password *

Confirm password

Register

© 2026 Fast Fuel. All rights reserved. [About](#)

5.2.2 Bejelentkezés

Welcome

Login

Register

Username

Enter username

Password

Enter password

Login

Don't have an account? Register

© 2026 Fast Fuel. All rights reserved. [About](#)

5.3 Főmenü

Bejelentkezés után a felhasználó először ide érkezik. Itt van feltüntetve az összes opció, ahova a felhasználó tud navigálni a gombok használatával. Szerepkörtől függ, hogy milyen oldalak láthat a felhasználó, ezeket az admin felhasználó szerkesztheti, habár néhány specifikus oldalt (például “My Shifts”) csak bizonyos típusú felhasználóhoz rendelhet hozzá. Az átláthatóság kedvéért kategóriákba vannak osztva a menüpontok.

Home

Home

Admin User (admin)

Logout

USER MANAGEMENT

Admins

Customers

Employees

Machines

Roles

ME

Profile

ORDERS

Orders

RESTAURANT

Shifts

Station Categories

Stations

© 2026 Fast Fuel. All rights reserved. [About](#)

5.4 Menedzsment oldalak

A menedzsment oldalak nagyon hasonlóak, csak a szerkesztett entitás különbözik. Lehet hozzáadni, szerkeszteni és törölni rekordokat. A meglévő jogoktól függ, hogy a felhasználó módosíthatja-e (hozzáadás, szerkesztés, törlés, ezek mind külön-külön joghoz vannak kötve).

5.4.1 Az oldal áttekintése

[Home](#)

Ingredient Manager

Admin User (admin)

[Logout](#)

Ingredients

Create

Name	Allergies	Station Categories	Edit	Delete
Beef Patty	None	Burger Station	Edit	Delete
Bun	Gluten	Burger Station	Edit	Delete
Cheese	Dairy	Burger Station	Edit	Delete
Lettuce	None	Burger Station	Edit	Delete
Oil	None	Fries Station	Edit	Delete
Potato	None	Fries Station	Edit	Delete
Salt	None	Fries Station	Edit	Delete

© 2026 Fast Fuel. All rights reserved. [About](#)

5.4.2 Rekord felvétele

Új rekord felvételéhez kattintsunk a Create gombra.

Create ingredients

×

Name *

Image URL

Allergies

Select allergies

⌵

Station Categories

Select station categories

⌵

Default Timer Value (seconds) *

0

⌵

Cancel

Create

5.4.3 Rekord szerkesztése

Meglévő rekord szerkesztéséhez kattintsunk az Edit gombra.

Edit ingredients

×

Name *

Beef Patty

Image URL

Allergies

Select allergies

⌵

Station Categories

Burger Station ×

Select station categories

⌵

Default Timer Value (seconds) *

0

⌵

Cancel

Save

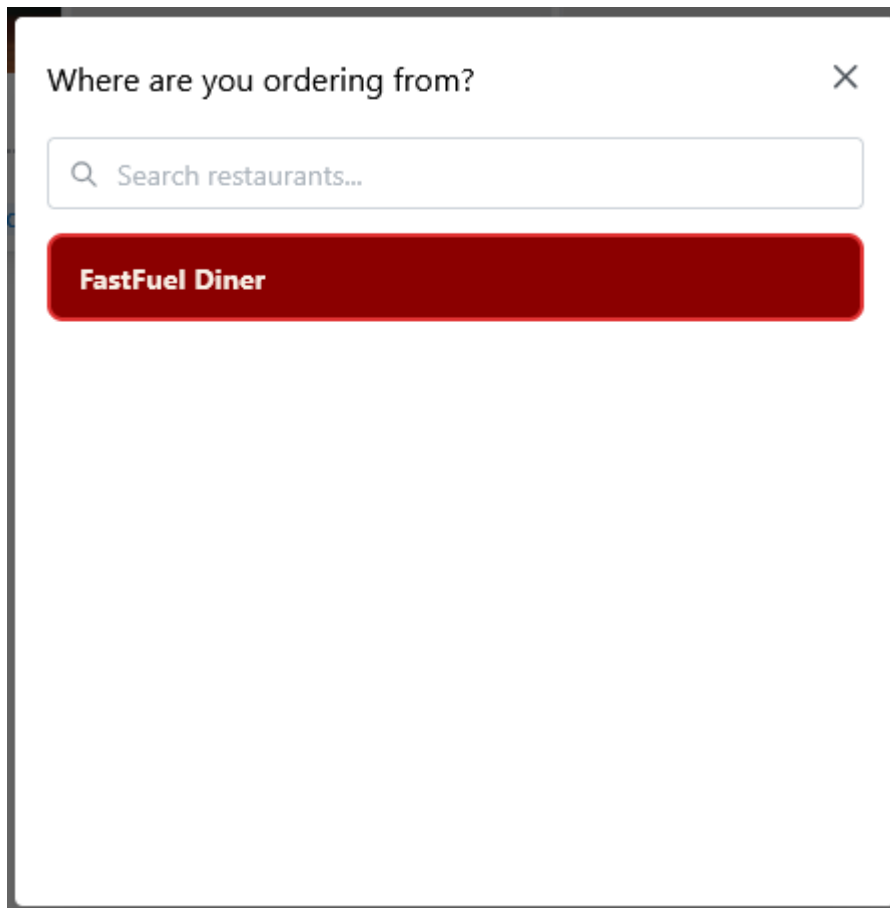
5.4.4 Rekord törlése

Meglévő rekord törléséhez kattintsunk a Delete gombra.

5.5 A rendelés folyamata

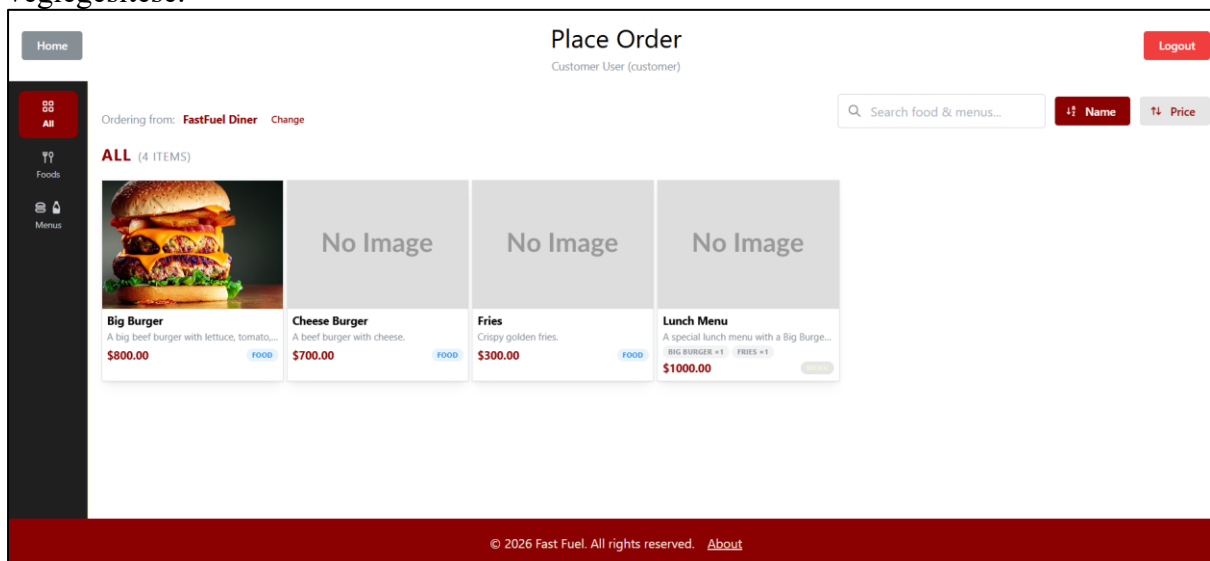
5.5.1 Étteremválasztó ablak

Ha a felhasználónak nincs beállítva a munkahelye (tehát nem employee vagy machine típusú), akkor a rendelés megkezdése előtt manuálisan választhat az éttermek közül. Egyéb esetben a munkahelye automatikusan kiválasztódik.



5.5.2 Étlap böngészése

A rendelésleadó felületnél különösen figyeltünk az átláthatóságra és felhasználóbarát kialakításra. Külön filtereket lehet beállítani a menükre és az ételekre. Keresési és rendezési funkció is elérhető. Miután az első ételt kiválasztottuk elérhetővé válik a rendelés véglegesítése.



5.5.3 Termék hozzáadása

Ha a felhasználó rákattint az egyik termékre akkor a rendszer bekéri a rendelni kívánt mennyiséget és hogy milyen speciális igénye van. A rendelés folyamán a felhasználó végig látja és szerkesztheti a kosár aktuális tartalmát. Amikor a felhasználó végzett a rendeléssel, a rendszer összegzi a rendelést és bekéri a bankkártyát vagy a készpénzt a fizetéshez.

Big Burger

A big beef burger with lettuce, tomato, and cheese.

Customise

Quantity

1

Special instructions

e.g. no onions, extra sauce...

\$800.00

Add to order

Your order

(1 items)

Cheese Burger

-

1

+

\$700.00

Total

\$700.00

Place order

Confirm your order

Ordering from: **FastFuel Diner**

Cheese Burger x1

Total

\$700.00

\$700.00

Back

Confirm & Pay

Payment

Total to pay:

\$700.00

Payment method

☒ Card ☐ Cash

Card details

Card number

1234 5678 9012 3456

Expiry date

MM/YY

CVV

123

Pay \$700.00

Back

Order placed!

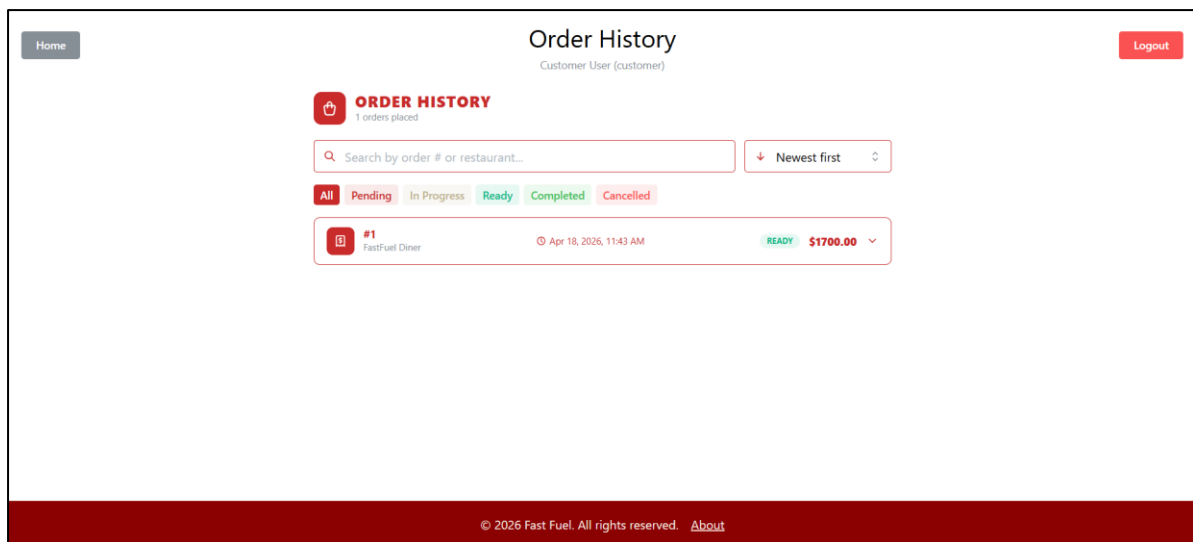
Order #2

Sit back and relax - your food is on its way!

OK

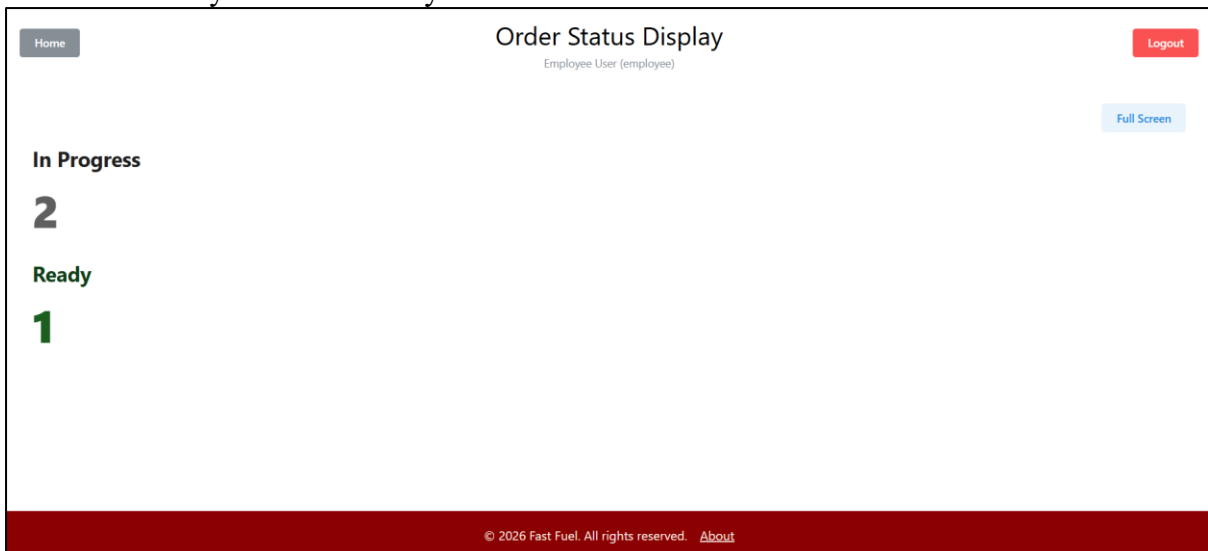
5.5.4 Rendelési előzmények

A felhasználó meg tudja tekinteni a rendelési előzményeit, tudja őket szűrni állapot alapján és megtekinteni annak elemeit, illetve árát. Az adatok a rendelés pillanatában érvényben lévő állapotot tükrözik.



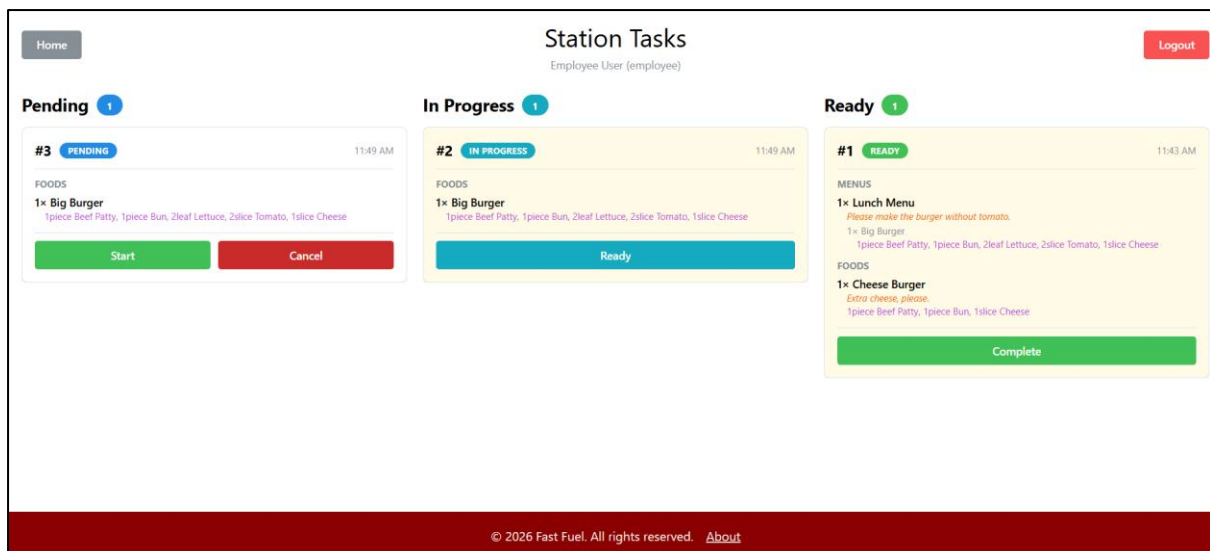
5.5.5 Rendelés státusz oldal

Ez az oldal a lobby-ban elhelyezett kijelzőknek van szánva. Egyszerűen meg lehet nézni melyik rendelés van folyamatban és melyik van kész.



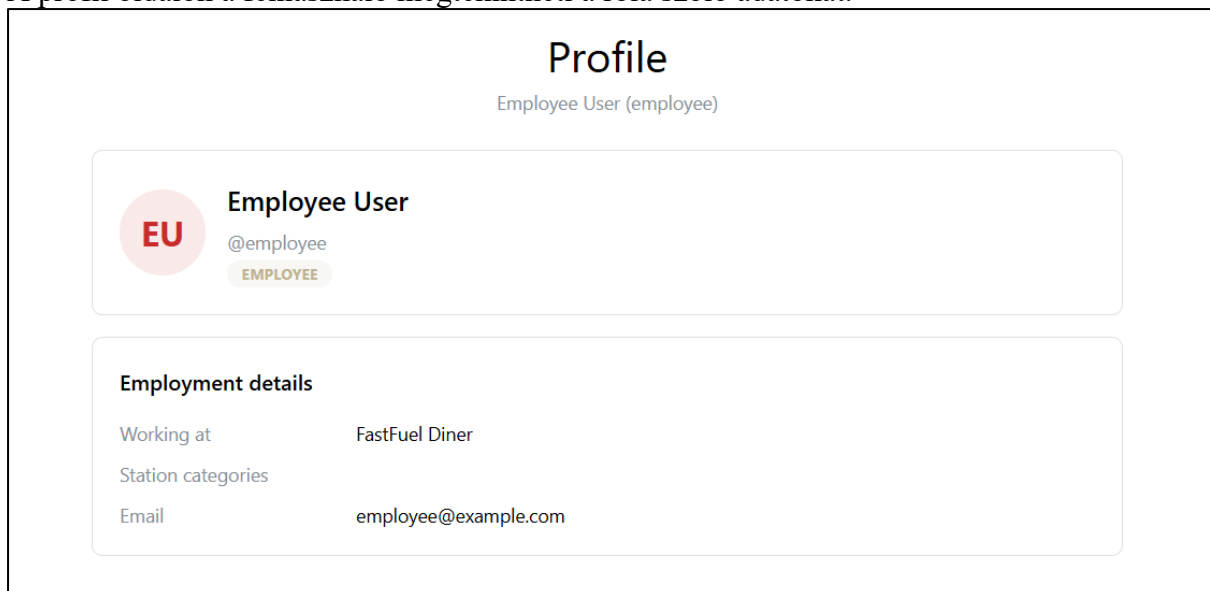
5.5.6 Egy állomás elvégzendő feladatai

A rendszer 3 kategóriába osztja a rendeléseket (amint a lenti képen is látható). Az elkészítendő sorban a rendelést el lehet fogadni vagy ki lehet törölni. Az elfogadott rendelések az elkészítésben levő oszlopba kerülnek, ahonnan az elkészítést követően a kész oszlopba kerülnek. Ezek az állapotok élőben frissülnek a lobby kijelzőjén.



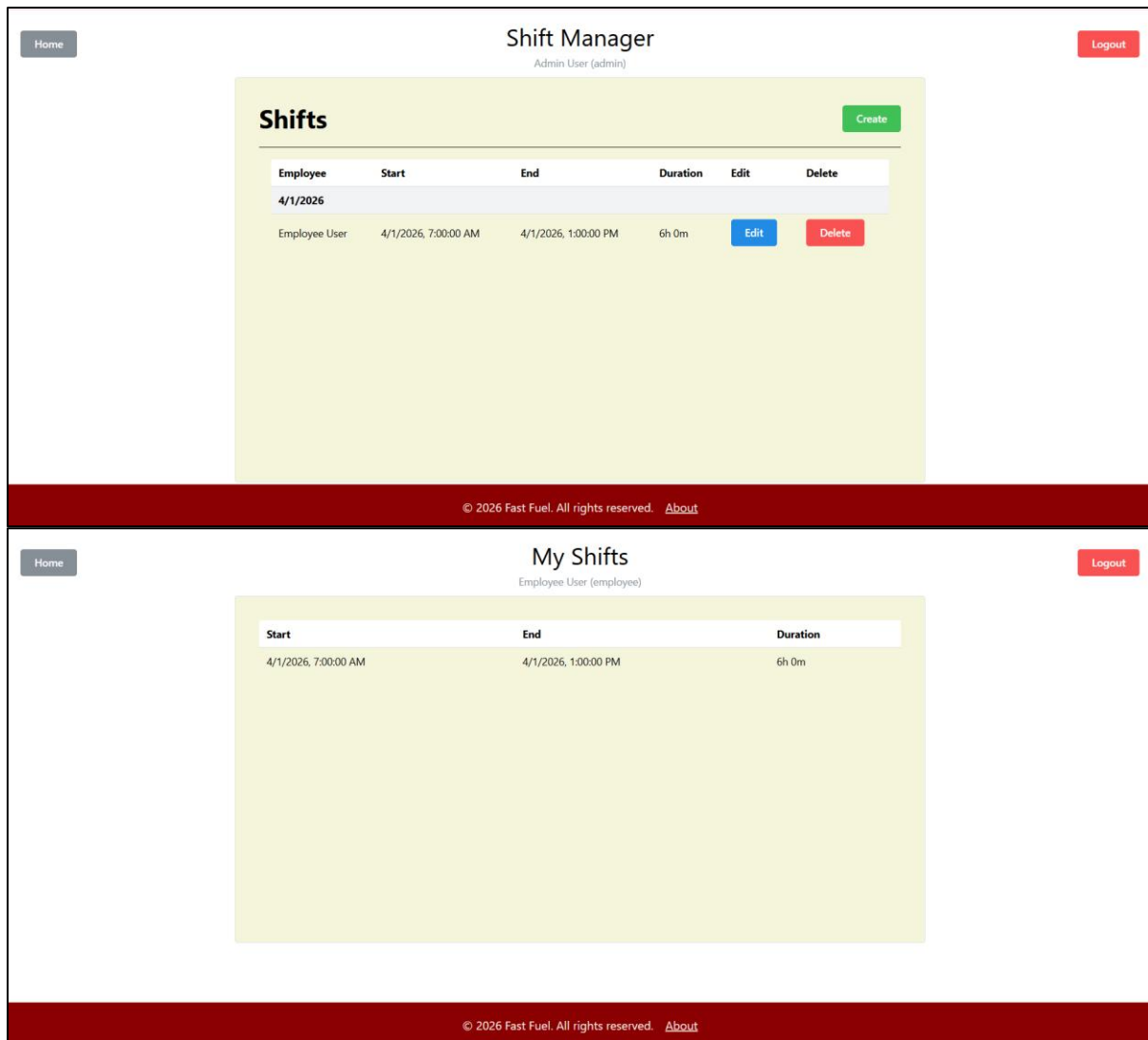
5.6 Profil oldal

A profil oldalon a felhasználó megtekintheti a róla szóló adatokat.



5.7 Műszakok

Admin felhasználónak (vagy akinek van rá jogosultsága) fel tudja venni az alkalmazott műszakjait, amit aztán az alkalmazott megnézhet a saját oldalán.



5.8 Jogosultságkezelés

Admin felhasználóként lehetőségünk van felhasználókat szerepkörökbe rendezni. Minden felhasználótípushoz tartozik egy beépített, alapértelmezett szerepkör, de újakat is csinálhatunk. A szerepkörökhöz jogosultságokat rendelhetünk és meghatározhatjuk, hogy milyen oldalakat lásson a főmenüben.

Admin felhasználóként lehetőségünk van felhasználókat szerepkörökbe rendezni. Minden felhasználótípushoz tartozik egy beépített, alapértelmezett szerepkör, de újakat is csinálhatunk. **A szerepkörökhöz jogosultságokat rendelhetünk és meghatározhatjuk, hogy milyen oldalakat lásson a főmenüben.**

Create role X

Name *
Shift Managers

Permissions
Permission:Shift:Read X Permission:Shift:Create X Permission:Shift:Update X
Permission:Shift:Delete X Permission:Employee:Read X Search permissions...

Users
Shift Manager User (Employee) X Search users...

Displayed Pages

Admin

MISSING REQUIRED PERMISSIONS

Required: Permission:Admin:Read
Recommended: Permission:Admin:Create, Permission:Admin:Update, Permission:Admin:Delete

Add Necessary Permissions Add All Permissions

Allergies

MISSING REQUIRED PERMISSIONS

Required: Permission:Allergy:Read
Recommended: Permission:Allergy:Create, Permission:Allergy:Update, Permission:Allergy:Delete, Permission:Allergy:Read

Add Necessary Permissions Add All Permissions

Create Order

MISSING REQUIRED PERMISSIONS

Required: Permission:Order:Create, Permission:Order:Read, Permission:Menu:Read
Recommended: -

Add Necessary Permissions Add All Permissions

Roles

Create

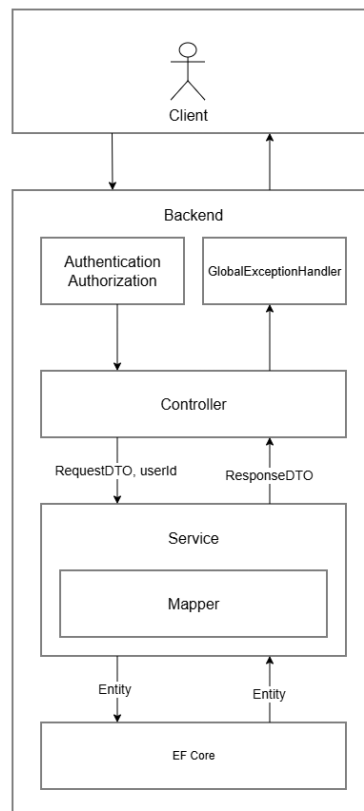
Name	Built-in?	Edit	Delete
Customer	Yes	Edit	Delete
Employee	Yes	Edit	Delete
Machine	Yes	Edit	Delete
Admin	Yes	Edit	Delete

6 A backend technikai felépítése

Mappaszerkezet:

```
+---FastFuel
|   +---Features
|   |   +---Common
|   |   |   +---Controllers
|   |   |   +---DbContexts
|   |   |   +---Exceptions
|   |   |   |   \---AppExceptions
|   |   |   +---Interfaces
|   |   |   +---Permissions
|   |   |   \---Services
|   |   |       \---CrudOperations
|   |   +---Feature1
|   |   |   +---Controllers
|   |   |   |   \---DTOs
|   |   +---Feature2
|   |   |   +---Common
|   |   |   +---Configs
|   |   |   +---Controllers
|   |   |   +---DTOs
|   |   |   +---Entities
|   |   |   +---Mappers
|   |   |   \---Services
|   +---NSwag
|   +---CrudOperationSummary
|   +---MarkAsRequiredIfNotNullable
|   +---PermissionSchema
|   +---SwaggerQueryParam
|   \---UnregisteredStatusCodeResultOperation
```

Pipeline:



A backend tervezésénél elsődleges szempont volt a bővíthetőség, kevés kódismétlés, rendszerezettség, átláthatóság, egységesség és a best practice-k követése.

6.1 Features mappa

A kódot feature-ök szerint rendszereztük, ezeken belül pedig további alkategóriák foglalhatnak helyet. A Common mappában található a megosztott kódok. A fontosabb *feature* mappákat külön is tárgyaljuk.

6.1.1 Features/feature mappa

Egy *feature* mappához általában (de nem szükségszerűen) tartoznak API végpontok a CRUD műveletekkel, az ehhez szükséges üzleti logikával és adatmodellekkel. A kódokat az alábbi alkategóriákba rendszereztük:

6.1.1.1 Entities

Adatbázisentitások. Reprezentál egy adatbázisban található táblát.

6.1.1.2 Configs

Az entitásokhoz tartozó konfigurációk. Ehhez Fluent API-t használtunk.

6.1.1.3 DTOs

Az API-n keresztül történő adatfogadásra és adatküldésre alkalmas objektumok.

6.1.1.4 Mappers

Az entitások és a DTO-k közötti konvertálásra szolgáló osztályok.

6.1.1.5 Controllers

Az API végpontokért felelős osztályok.

6.1.1.6 Common

Más kategóriába nehezen sorolható objektumok, ezeket általában több rétegből is használjuk

6.1.2 Features/Common mappa

Ebben a mappában találhatók azok a kódok, amiket több feature is használ pl. absztrakt osztályok, interfészek, exception-ök.

6.1.2.1 Interfaces

Itt találhatóak az egész alkalmazásban gyakran használt interfészek, melyeket más mappába nem tudtunk sorolni.

Identifiable: biztosítja, hogy egy osztály rendelkezzen egyedi azonosítóval. Erre az absztrakt, generikus osztályoknak van szüksége, mert ezek az osztályok nem tudják, hogy pontosan milyen struktúrájú adatokkal fognak dolgozni, ennek ellenére szükségük van egy azonosítóra, mert például ez alapján törölnek az adatbázisban, vagy ez alapján tájékoztatják a webklienst, hogy hol jött létre egy új resource (201 Created).

IMapper: egy "szerződés", amely leírja, hogy az adott osztálynak képesnek kell lennie az API-n bejövő adatot konvertálni adatbázisentitássá, valamint ugyanezt a másik irányba is megtenni. Ezen kívül képesnek kell lennie meglévő entitások frissítésére bejövő adatok alapján. Ezt az interfészt használják a service-k dependency injection-hoz.

6.1.2.2 Services

ICrudService: hasonló interfész az IMapper-hez. Biztosítja azt, hogy egy osztály képes DTO-okon keresztül kommunikálva lekérdezni, létrehozni, frissíteni vagy törölni (tulajdonképpen a CRUD műveletek, innen a neve) adatokat.

CrudService: generikus implementáció az ICrudService interfészre. A legtöbb feature-nek ez egy teljes egészében megfelelő implementáció, azonban felépítése rugalmas, lehetővé teszi a műveletekhez tartozó egyes lépések módosítását, például: entitás lekérdezése, DTO - entitás konverzió, mentés az adatbázisba. Ehhez a **CrudOperations** mappában található, generikus megoldást nyújtó osztályokat kell felülrni és a felülírt osztályból egy példányt biztosítani a CrudOperation leszármazott osztályának. Jó példa erre a jelszó hash-elése, ahol a mentés lépés van úgy felülírva úgy, hogy előbb módosítsa az entitásban a jelszót a megfelelő módon.

6.1.2.3 Controllers

A **CrudController** a CrudService-hez hasonló absztrakt generikus controller, az API végpontokért felelős. Ez az alap implementáció is legtöbbször teljesen elégséges a leszármazott osztályoknak, de biztosít lehetőséget módosításra.

6.1.2.4 Permissions

Itt vannak a jogosultságkezeléshez kapcsolódó attribútumok: **PermissionCheck** és **SkipPermissionCheck**, valamint az ezekhez kapcsolódó logika. A PermissionCheck 1 paramétert vár: a művelet nevét. Ez alapján, valamint reflection-el rakja össze a permission nevét: `Permission:Entitásnév:Műveletnév`. Például: `Permission:Restaurant:Create`. A SkipPermissionCheck öröklődésnél hasznos, amikor az őszosztály egy metódust megjelöl PermissionCheck-el, de a leszármazott osztály publikussá akarja tenni. Ugyanezt meg tudjuk tenni a beépített AllowAnonymous-sal, de az az autentikációt is feloldja. A filterezést a PermissionFilter végzi.

6.1.2.5 DbContexts

A **FastFuelDbContext** biztosítja az adatbáziskapcsolatot. Az felhasználók entitásainál öröklődést is használunk, ennek a tárolására TPT (Table Per Type) stratégiát használunk. Az kivételek feldolgozásához és az alapértelmezetten túlságosan általános hibaüzenetek kiküszöbölésére a EntityFramework.Exceptions.MySQL.Pomelo könyvtárat használjuk.

6.1.2.6 Exceptions

Az **AppException** leszármazott osztályai azokat a hibákat hivatottak jelezni amire az alkalmazás fel van készülve, de a hibát a kliensnek továbbítani szükséges. Erre jó alternatív megoldás lett volna a Result Pattern, amit sajnos idő hiányában nem implementáltunk. Minden AppException-höz tartozik egy Http kód, amit a **GlobalExceptionHandler** rendel hozzá az exception-okhoz és ez alapján küld választ a request-re.

6.1.3 Features/Auth mappa

Az autentikációhoz használt kód a Features/Auth mappában található. A kód nagyrészt megegyezik az alapértelmezetten ASP.NET Identity-vel szállított implementációval, azonban az interneten is ismert probléma, hogy nehezen módosítható vagy egészíthető ki, ezért használunk saját implementációt. A kliens választhat Bearer és Cookie alapú válasz séma között.

6.1.4 Features/Permissions mappa

Ez a szolgáltatás két egyszerű feladatot lát el: lekérdezni az összes, valamint a bejelentkezett felhasználó jogosultságait.

6.1.5 Features/Pages mappa

Mivel a projekt egyik alapeleme, hogy szerepkörtől függően az admin **módosítani** tudja a navigációs menüben megjelenített oldalakat, elkerülhetlenné vált, hogy erre a backend logikát is felkészítsük, de az eltárolt információt igyekeztünk minimálisra szorítani, hiszen ez a frontend-re tartozó dolog. Szükség volt eltárolni a létező oldalakat, valamint az egyes oldalak funkcionálásához szükséges jogosultságokat. Utóbbit két kategóriába szedtük: kötelező és ajánlott. Előbbi az adott oldal működéséhez elengedhetetlen, míg utóbbi csak új gombokat, adatokat, egyszóval funkcionalitást ad hozzá. Ezek az információk automatikusan rögzítésre kerülnek az OpenAPI dokumentációban, valamint az API-n keresztül lekérdezhetőek is. A frontend development környezetben ebből **automatikusan TypeScript típusok készülnek**, melyekkel biztosítjuk, hogy egy oldal **nem próbál meg olyan műveletet végrehajtani, amihez a szükséges jogosultsága a nyilvántartás szerint nincs** és felkészül azokra az esetekre amikor egy ajánlott jogosultsággal egy felhasználó nem rendelkezik.

Vannak olyan oldalak, amiknek nem elég bizonyos jogosultságok megléte, hanem függenek az felhasználó típusától. Például egy vásárlónak hiába adjuk meg a jogot, hogy lekérje a műszakokat, attól neki még nem lesz egy műszakja sem, így számára a “saját műszakok” oldal használhatatlan. Ezért ezeknél az oldalaknál nyilván van tartva, hogy milyen felhasználótípushoz rendelhetőek csak hozzá (Pontosabban, hogy milyen beépített jogosultság szükséges hozzá, de ezeknek 1-1 hozzárendeltetésük van egymáshoz, szóval nem igazán számít. Erről bővebben a következő a következő fejezetben olvashatunk).

6.1.6 Features/Roles mappa

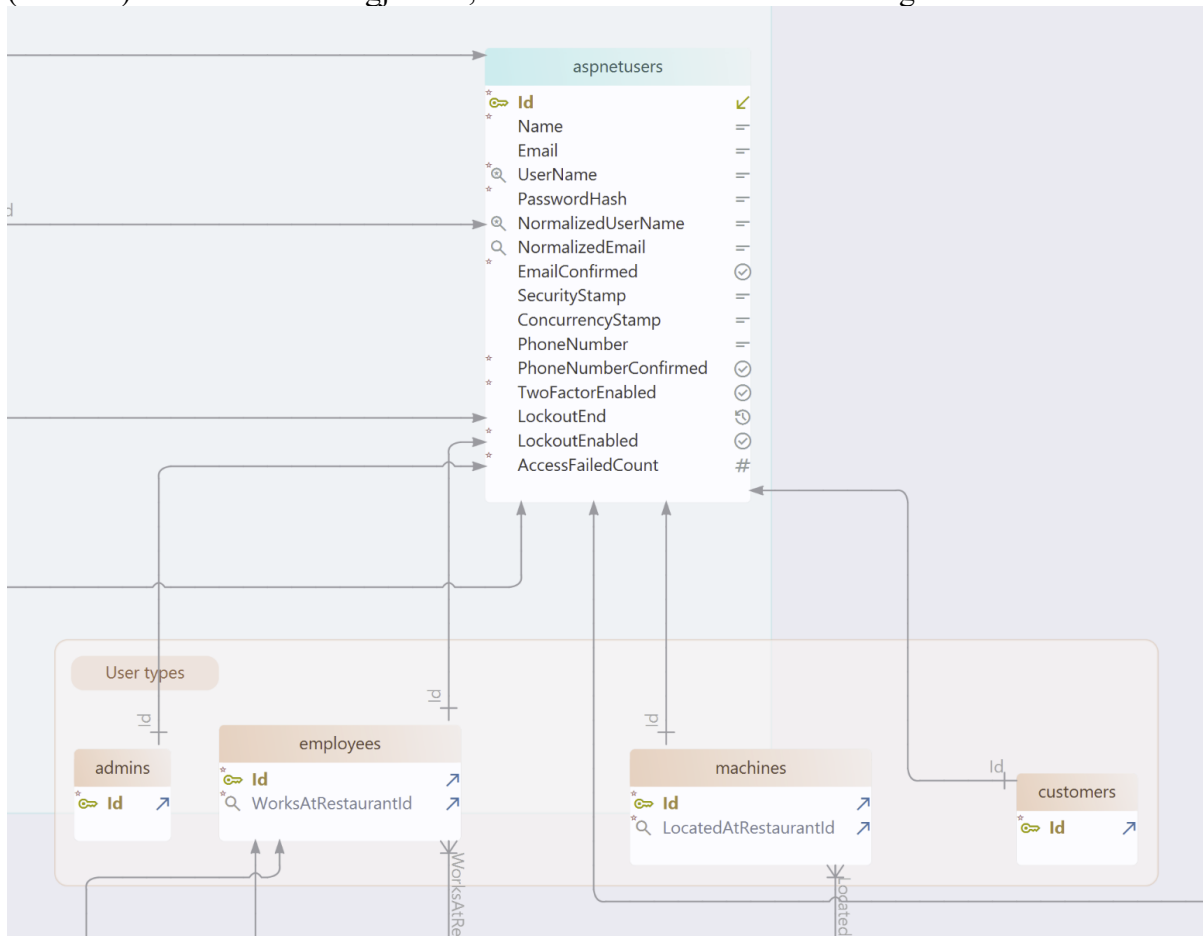
Itt lehet kezelni a szerepköröket a klasszikus CRUD műveletekkel. Minden szerepkörhöz tartoznak jogosultságok, valamint a navigációs menüben számára látható oldalak. A 4 felhasználó típushoz tartozik 4 beépített szerepkör (természetesen akárhány létrehozható, csak azok nem kerülnek automatikusan hozzárendelésre). Ezek automatikusan hozzárendelésre kerülnek az adott típusú felhasználó regisztrációjakor. Nevük és a hozzárendelt felhasználók nem módosíthatóak és nem törölhetőek. Ezt a különleges státuszt az **IsDefault** tulajdonsággal jelezzük. Ezen 4 beépített szerepkör közül is az admin a legspeciálisabb, mivel ő kötelező módon rendelkezik az összes jogosultsággal. Ezt az **ArePermissionsImmutable** tulajdonsággal jelezzük.

6.1.7 Features/Users mappa

Itt található az a logika, ami minden felhasználótípushoz szükséges, valamint 2 endpoint: GET /User és GET /User/me. Előbbi arra szolgál, hogy felhasználótípustól függetlenül lekérdezzük az összes felhasználó, utóbbi pedig arra, amikor a kliens még nem tudja, hogy milyen felhasználótípus van bejelentkezve, ezért lekérdezi az alapadatokat. Ettől függően fog később kérést küldeni a /Admin/me, /Customer/me, /Employee/me vagy a /Machine/me egyikére. A 4 felhasználótípus innen fog öröklödni, extra adatmezőket hozzáadva az User-hez. Ennek tárolásáról a [DbContexts](#) fejezetben olvashatunk bővebben.

Az SQL táblát megnézve az Users táblában extra adatmezők is eltárolásra kerülnek, mivel az ASP.NET Identity könyvtár által szolgáltatott IdentityUser-ből örököl. Néhány mezőt pedig

ugyaneből az okból *new* kulcsszóval írunk felül létre, mivel az IdentityUser-ben opcionális (nullable) mezőként van megjelölve, nekünk azonban feltétlen szükségünk van rá.



6.1.8 Features/Orders mappa

A rendelésekhez kapcsolódó üzleti logika hasonló a generikus CrudController-hez, azonban attól annyi ponton eltér, hogy kénytelenek voltunk elhagyni az öröklődést. Egy ilyen eltérésre példa, hogy a rendelést nem lehet utólag szerkeszteni, így az a végpont nem szerepel a controllerben. Másik példa, hogy a GetAll végpont egyedi URL query paramétereket vár, ezek alapján lehet a rendeléseket szűrni.

Elképzeltető, hogy a rendelés leadása után a rendelésben szereplő termékek módosulnak, például megváltozik az árak vagy más nevet kapnak. Ezeknek az adatoknak a rendeléskor történő elmentésével azonban megakadályozzuk azt, hogy ez a rendelési előzményeket manipulálja.

6.2 NSwag OperationProcessor-ok

Az OperationProcessor-ok felelős azért, hogy a Swagger/OpenAPI dokumentáció generálása során egy-egy API-végpont (operation) metaadatait feldolgozza és módosítsa.

6.2.1 CrudOperationSummary

Mivel viszonylag sok végpont működése azonos (CrudController alapértelmezett viselkedés, lásd [Controllers](#)), a controller-el sokszor nem tartalmaznak saját metódus implementációt, ehelyett csak ennyiből állnak:

```
using FastFuel.Features.Common.Controllers;  
using FastFuel.Features.Common.Services;  
using FastFuel.Features.Foods.DTOS;  
using FastFuel.Features.Foods.Entities;
```



```
namespace FastFuel.Features.Foods.Controllers;

public class FoodController(ICrudService<FoodRequestDto, FoodResponseDto> service)
    : CrudController<Food, FoodRequestDto, FoodResponseDto>(service);
```

Előnye, hogy nincs kódismétlés és gyors fejlesztést tesz lehetővé, hátránya, hogy a dokumentációt az őszosztályból örökli. Mi azonban nem szerettünk volna ennyire általánosan fogalmazni dokumentációírás közben, szerettük volna, hogy a leírásba legalább az entitás neve belekerüljön. Ezt oldottuk meg ezzel az `OperationProcessor`-al: a leírásba `Reflection` segítségével beilleszti az entitás nevét:

POST `/api/Station` Creates a new station.

6.2.2 MarkAsRequiredIfNotNullable

Ezzel egy frontenden jelentkező problémát oldottunk meg: a TypeScript típusok az OpenAPI sémából kerülnek generálásra, azonban amikor azt próbálja eldönteni, hogy egy paraméter kötelező-e, akkor a `required` tömböt vizsgálja, amit az NSwag alapértelmezetten üresen hagy, az ebbe tartozó elemeket külön-külön jelölni kell. Mi azonban ez redundánsnak éreztük, hiszen mi akkor tekintünk egy elemet kötelezőnek, ha az nem nullable (persze a viselkedés érthető, sokszor mást jelent a hiányzó elem, mint a nullable, a mi esetünkben ez nem így van). Ez az osztály minden nem nullable elemet belerak a `required` tömbbe.

6.2.3 PermissionSchema

Ez a két osztály a jogosultságok dokumentációját segíti elő. A `PermissionSchemaOperationProcessor` az OpenAPI dokumentációban helyezi el az adott végponthoz szükséges jogot. A `PermissionSchemaDocumentProcessor` szintén az OpenAPI sémát módosítja: amikor valahová egy jogosultság nevét várjuk, akkor annak a lehetséges értékei ismertek, ezért az általános string típust "string literal"-ra cseréljük, ezzel is segítve frontend-en a kódgenerálást.

6.2.4 SwaggerQueryParam

Ez sajnos egy túl optimális nyújt a következő problémára: a controller-ek egy közös ősből örökölnék, aminek a signature-jét már nem módosíthatják. Azonban párszor szükség volt hozzáadni egy URL query-t például szűrésre. A megadott paramétereket még lehetséges kiolvasni a signature módosítása nélkül is, azonban ez nem kerül dokumantálásra. Ezt hivatott jelölni az attribútum és ennek segítségével módosítjuk az OpenAPI sémát a megfelelő módon.

```
[HttpGet]
[SwaggerQueryParam("status", typeof(OrderStatus))]
[SwaggerQueryParam("restaurantId", typeof(uint))]
[PermissionCheck(CrudOperation.Read)]
public override async Task<Results<Ok<List<OrderResponseDto>>,
    BadRequest<ProblemDetails>, UnauthorizedHttpResult,
    ForbidHttpResult>> GetAll(
    CancellationToken cancellationToken = default)
{
    ...
}
```

6.2.5 UnregisteredStatusCodeResultOperation

Automatikusan kiegészíti az OpenAPI dokumentációt azokkal az HTTP státuszkódokkal, amelyeket az NSwag beépített pipeline-ja egyébként kihagyna (pl. `ForbidHttpResult`). Rekurzívan bejárja az endpoint visszatérési típusát, és minden olyan osztályhoz, amely `IStatusCodeHttpResult`-ot implementál, felveszi a megfelelő státuszkódot az OpenAPI leírásba.

6.3 Környezeti változók

Az adatbázis elérését, valamint a Development/Production üzemmódot környezeti változók segítségével tudjuk konfigurálni. Fejlesztés közben ehhez .NET User Secret-eket használtunk, az éles build-ben pedig Dockeren keresztül állítottuk be a szükséges értékeket. Erről bővebben az Üzembehelyezési Útmutató [Konfiguráció](#) fejezetében olvashatunk.

6.4 Program.cs

Itt kapcsolódik össze minden, Dependency Injection segítségével. Konfiguráljuk az autentikációt, az adatbázist, automatikus szkenneléssel hozzárendeljük az interfészekhez a konkrét implementációt, valamint definiáljuk mi történjen hiba esetén. Miután ezek a szolgáltatások rendelkezésre állnak felállítjuk a bejövő kéréseket kiszolgáló pipeline-t. Végül feltöltjük az adatbázist, ami éles környezetben csak egy alap admin felhasználót jelent, fejlesztési környezetben azonban tesztadatokat is hozzáadunk.

```
private static async Task MainAsync(string[] args)
{
    var builder = WebApplication.CreateBuilder(args);

    // Configure authorization and authentication (JWT)
    ConfigureAuth(builder);

    // Configure database
    ConfigureDatabase(builder);

    // Configure other application services (controllers, OpenAPI, CORS, DI)
    ConfigureAppServices(builder);

    // Configure global exception handlers
    AddExceptionHandler(builder);

    var app = builder.Build();

    // Configure middleware / request pipeline
    ConfigureMiddleware(app);

    // Seed the database with initial data (roles, users, etc.)
    await SeedDatabaseAsync(app);

    await app.RunAsync();
}
```

7 A frontend technikai felépítése

A frontend fejlesztéséhez Vite-ot, React-ot és TypeScript-et használtunk. Elsődleges szempont volt a minél erősebb típusosság (a hibák elkerülése végett) és a fejlesztési sebesség maximalizálása. Előbbihez nyílt forráskódú és saját kódgenerációs megoldást is alkalmaztunk, utóbbihoz pedig a projekt elején igyekeztünk minél több és rugalmasabb újrahasznosítható komponenszt írni.

7.1 Általános, újrahasznosítható komponensek

7.1.1 NumericCombobox

Nagyon egyszerű komponens: a Mantine nem támogatja a szám típusú kulccsal rendelkező combobox-okat, ezért csináltunk egy wrapper-t, az eredeti komponensnek minden funkcionalitását megtartva

7.1.2 Paper

A Mantine által exportált Paper személyre szabott változata.

7.1.3 PagePermissionGuard

Egy page komponens és annak nevét várja paraméterként. Biztosítja, hogy az oldal csak akkor tölthet be, ha a felhasználó rendelkezik legalább a funkcionáláshoz szükséges minimális jogosultságaival. Erről bővebben a backend [Pages](#) fejezetében olvashatunk.

7.1.4 SearchablePickerModals

A SearchablePickerModal egy generikus elem-kiválasztó modal, amit bármilyen típusú listához be lehet kötni. Használja a RestaurantPickerModal és a StationPickerModal.

7.2 EntityManager

Az EntityManager a projekt egyik legkomplexebb komponense. A legtöbb menedzseroldal ennek a használatával készült. A táblázat és a szerkesztőfelület logikai felépítését elég JavaScript objektumként megadni és ezekből építkezve a vizuális megjelenítést, valamint az állapotkezelést önállóan kezeli. Két fő alkomponensből áll: EntityEditor és EntityTable. Értelmszerűen előbbi a szerkesztőért, míg utóbbi a táblázatért felel.

7.2.1 EntityTable

A példán jól látszik, hogy minimális kódmennyiséggel lehet egy teljesen működő táblázatot létrehozni:

```
export function exampleComponent() {
  const data = useExampleData();

  const tableColumns: ColumnDefinition<Food>[] = [
    { header: 'Name', accessor: 'name' },
    { header: 'Price', accessor: 'price' },
    { header: 'Description', render: (food) => getDisplayedDescription(food.description) },
    {
      header: 'Image',
      render: (food) =>
        food.imageUrl ? (
          <Image src={food.imageUrl} alt={food.name} width={50} height={50} />
        ) : (
          'No image'
        ),
    },
  ];

  return (
```

```

    <EntityTable
      data={data}
      columns={tableColumns}
      onEdit={(item) => console.log(item)}
      onDelete={(item) => console.log(item)}
    />
  );
}

```

Jól látszik, hogy minimális kódmennyiséggel lehet egy teljesen működő táblázatot létrehozni. Ezen kívül számos más paraméterrel is személyre szabható, amelyek azonban a kódot olvasva magától értetődőek és a tömörség kedvéért itt nem tárgyaljuk.

Foods

Create

Name	Price	Description	Image	Ingredients	Edit	Delete
Big Burger	800	A big beef burger with lettuce, tomato, and cheese.		Beef Patty, Bun, Lettuce, Tomato, Cheese	Edit	Delete
Cheese Burger	700	A beef burger with cheese.	No image	Beef Patty, Bun, Cheese	Edit	Delete
Fries	300	Crispy golden fries.	No image	Potato, Salt, Oil	Edit	Delete

7.2.2 EntityEditor

A táblázatban az Edit és a Create gombra kattintás hatására egy szerkesztőablak ugrik fel. A táblázathoz hasonlóan a felépítését egy objektummal határozhatjuk meg. Számos típusú mező szerkesztését támogatja az egyszerű szövegektől és számoktól kezdve, a címkéken át, a szótárakat tartalmazó listáig. Ha ez mégsem lenne elég, a táblázathoz hasonlóan egyedi mezőket is létrehozhatunk egy komponenssel visszatérő callback segítségével. Szabályozhatjuk továbbá, hogy egy mező mikor legyen kötelező, illetve nullable. Jó példa erre a jelszó, ami létrehozáskor kötelező, szerkesztéskor azonban elhagyható, ha nem kívánjuk változtatni. Ilyenkor a required tulajdonság „create” (akkor lesz required amikor létrehozunk egy rekordot), míg a nullable „edit” (akkor lesz nullable amikor szerkesztünk). Fontos volt ezt a két tulajdonságot külön szabályozni, hiszen előfordult olyan szituáció, hogy egy mező lehet üres (tehát nem required), de az API nem fogad el null-t. Lehetőség van továbbá az adatok validálására, erre azonban a példa nem terjed ki. A komponens használó oldalak az EntityManagers mappában találhatóak.

```

import { useState } from 'react';
import { EntityEditor } from './EntityEditor';

const fields = [
  { type: 'text', key: 'name', label: 'Name', initialValue: '', required: 'always', nullable: 'never' },
  { type: 'password', key: 'password', label: 'Password', initialValue: '', required: 'create', nullable: 'edit' },
  { type: 'select', key: 'role', label: 'Role', initialValue: 'user', nullable: 'never',
    fieldProps: { data: ['user', 'admin'] } },
  { type: 'bool', key: 'active', label: 'Active', initialValue: true, nullable: 'never' },
  { type: 'list', key: 'tags', items: [
    { type: 'text', key: 'value', label: 'Tag', initialValue: '', nullable: 'never' },
  ] },
];

const editValues = { name: 'Jane', password: '', role: 'admin', active: true, tags: [{ value: 'vip' }] };

```

```

export const Example = () => {
  const [opened, setOpened] = useState(false);
  const [mode, setMode] = useState<'create' | 'edit'>('create');

  return (
    <>
      <button onClick={() => { setMode('create'); setOpened(true); }}>Create</button>
      <button onClick={() => { setMode('edit'); setOpened(true); }}>Edit</button>

      <EntityEditor
        opened={opened}
        onClose={() => setOpened(false)}
        title="User"
        fields={fields}
        onSubmit={(values, mode) => { console.log(mode, values); setOpened(false); }}
        {...(mode === 'edit' ? { mode, values: editValues } : { mode })}
      />
    </>
  );
};

```

Edit restaurant

×

Name *

FastFuel Diner

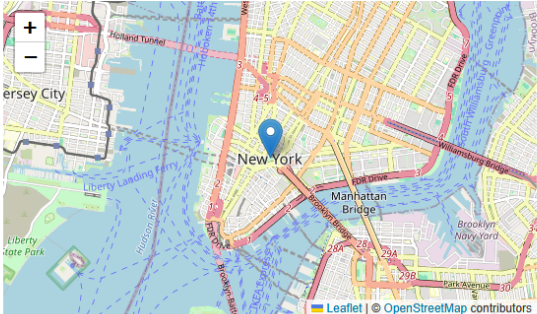
Description

A fast food restaurant serving burgers and fries.

Phone

555-1234

Location



Address *

123 Main St. Anytown, USA

Opening Hours

Day of Week *

Open Time (HH:mm) *

Close Time (HH:mm) *

Sunday

09:00

21:00

🗑️

Monday

09:00

21:00

🗑️

Add

Cancel

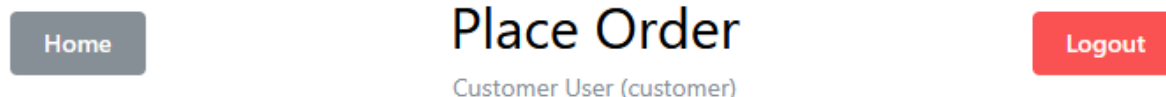
Save

7.3 Layout

Outlet segítségével header-t és footer-t ad az oldalakhoz, melyeket dinamikusan el lehet tüntetni, ha egy oldalnak erre szüksége van (pl., ha teljes képernyőre szeretnénk kitenni a megjelenített oldalt).

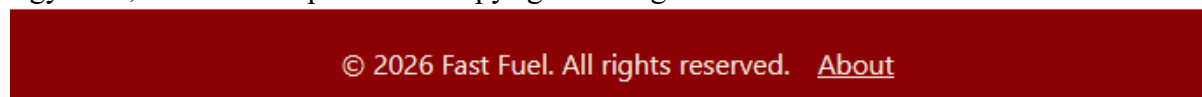
7.3.1 Header

A header tulajdonságai dinamikusan változtathatóak. Választhatunk, hogy jobb oldali gomb Login, Logout vagy Register legyen, valamint megadhatjuk az oldal címét. A bejelentkezett felhasználónak a neve, illetve a típusa automatikusan történik lekérdezésre.



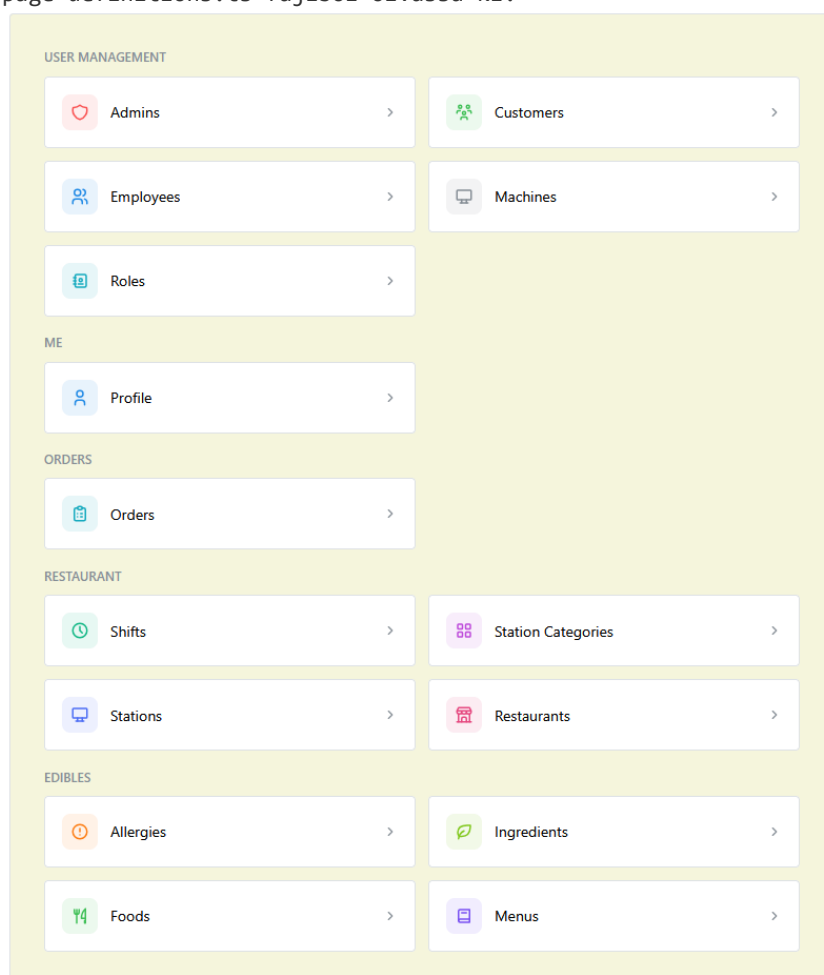
7.3.2 Footer

Egyszerű, statikus komponens. A copyright mindig frissül az aktuális évre.



7.4 Főmenü

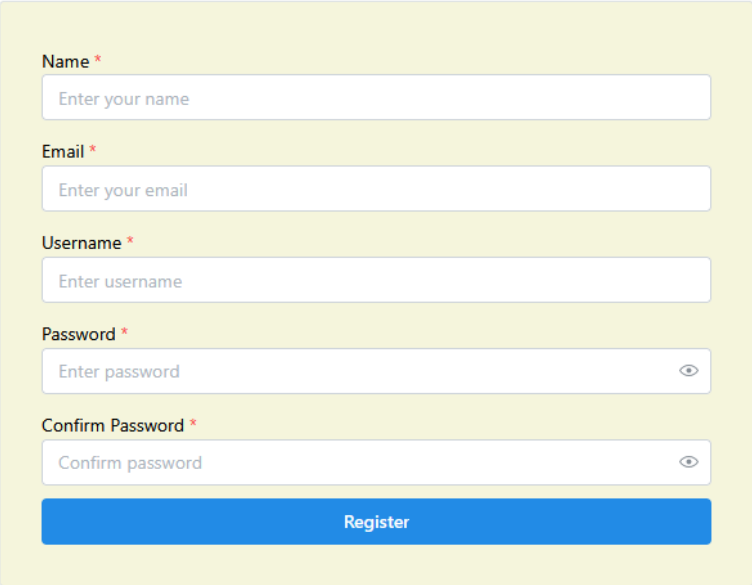
Csakis azokat a menüpontokat jeleníti meg, ami a bejelentkezett felhasználó rangjához konfigurálva van. Az egyes menüpontokhoz tartozó konfigurációt (cím, ikon, kategória) a `page-definitions.ts` fájlból olvassa ki.



7.5 Regisztráció

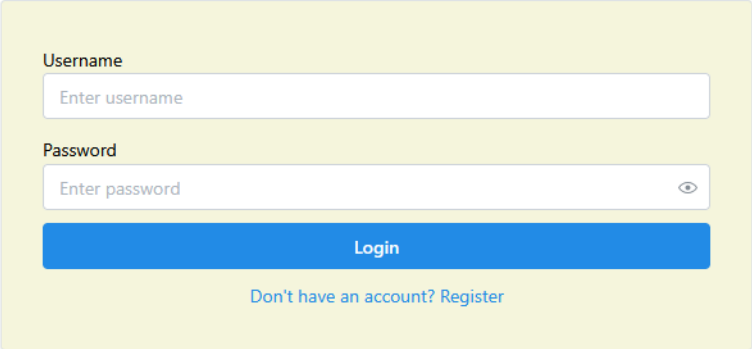
A regisztráció felület vásárlóknak lett szánva (minden más típusú felhasználót a megfelelő jogosultsággal rendelkező felhasználó – jellemzően admin – regisztrálhat), ennél fogva az űrlap kitöltése után egy POST kérést küld a `/api/Customer` végpontnak. Ha ez sikeres volt, kérünk egy token a `/api/Auth/login` végponttól. Query paraméterek segítségével jelezzük a

szervernek, hogy a token-t http-only cookie-kban szeretnénk tárolni, melyet a böngésző automatikusan végez, ezzel további teendő nincs. Sikeres regisztráció után a főmenübe navigáljuk a felhasználót.

A registration form with a light yellow background. It contains five input fields: 'Name *', 'Email *', 'Username *', 'Password *', and 'Confirm Password *'. Each field has a placeholder text: 'Enter your name', 'Enter your email', 'Enter username', 'Enter password', and 'Confirm password' respectively. The 'Password' and 'Confirm Password' fields have an eye icon to toggle visibility. At the bottom is a blue 'Register' button.

7.6 Bejelentkezés

A regisztrációval azonos módon működik, azzal a kivétellel, hogy itt bármilyen felhasználótípus be tud jelentkezni.

A login form with a light yellow background. It contains two input fields: 'Username' and 'Password'. The 'Username' field has a placeholder 'Enter username'. The 'Password' field has a placeholder 'Enter password' and an eye icon to toggle visibility. Below the fields is a blue 'Login' button. At the bottom, there is a link: 'Don't have an account? Register'.

7.7 Kódgenerálás

A kódgenerálást az `npm run generate` paranccsal futtathatjuk manuálisan, automatikusan pedig a dev szerver indulásakor fut le.

7.7.1 Kódgenerálás az OpenAPI séma alapján

Az OpenAPI sémából az [openapi-typescript](#) nevű könyvtárral typescript típusokat generálunk, ezzel biztosítva azt, hogy minden végpont helyesen megkapja az általa elvárt mezőket, illetve ismert lesz a válaszban szereplő mezők és azoknak típusai.

7.7.2 Kódgenerálás a jogosultságokhoz

Mint a [Pages](#) fejezetben már említettük, hogy egy oldalt jellemezni lehet azzal, hogy milyen jogosultságok elengedhetetlenek a működésükhöz, illetve milyen opcionális jogosultságok adhatnak hozzá extra funkciót. Ezek nyilvántartása közben nagy a hibázási lehetőség, hiszen elég egy sietve megírt http kérés, ami nem lett ledokumentálva és az alkalmazás nem fogja megkövetelni a működéshez szükséges jogosultságok meglétét amikor egy oldalt hozzáadunk egy szerepkör főmenüjéhez. Ha ezután a szerepkörbe tartozó felhasználó megpróbál hozzáférni az említett oldalhoz, 403-as hibába ütközhet.

Ezt a hibalehetőséget kívántuk elkerülni azzal, hogy létrehoztunk egy olyan hook-ot, aminek az oldal nevét megadva visszakapjuk a **dokumentált** végpontokat elérő hook-okat. Amennyiben nem létező oldalt, az adott oldalhoz nem dokumentált jogosultságot, vagy az adott jogosultsághoz nem tartozó végpontot adunk meg, a TypeScript hibát fog jelezni.

Opcionális visszatérési típusokkal (felveheti az undefined értéket) jelöli meg az olyan hook-okat, melyekhez tartozó jogosultságok az adott oldal funkcionálásához nem feltétlen szükségesek (ám ajánlottak). Így kényszeríti a fejlesztőt annak a scenáriónak a kezelésére, ha a felhasználó nem rendelkezik az adott jogosultsággal.

Ezt a viselkedést TypeScript típusok generálásával érjük el. *Nézzük végig lépésről lépésre, hogyan épül fel a generált fájl (page-permissions.generated.ts).*

Legfelül találhatóak az import-ok, ezekre még szükség lesz.

```
import createClient from 'openapi-react-query';
import type { paths } from './api-schema.generated.ts';
```

Ezt követi az összes oldal neve felsorolva. Később ezzel biztosítjuk, hogy a fejlesztő által megadott oldal neve valóban létezzen.

```
export type PageName =
  | "StationTasks"
  | "OrderStatusDisplay"
  | "OrderCreator"
  | "OrderHistory"
  // ...
;
```

Ezután felsoroljuk az összes létező jogosultság nevét.

```
export type PermissionName =
  | "Permission:Admin:Create"
  | "Permission:Admin:Delete"
  | "Permission:Admin:Read"
  | "Permission:Admin:Update"
  // ...
;
```

Majd minden oldalhoz hozzárendeljük a kötelező jogosultságokat.

```
export type PageNecessaryPermissions = {
  StationTasks: "Permission:Station:ViewTasks";
  OrderStatusDisplay: "Permission:Order:Read";
  OrderCreator: "Permission:Order:Create" | "Permission:Food:Read" |
  "Permission:Menu:Read";
  OrderHistory: "Permission:Food:Read" | "Permission:Menu:Read";
  // ...
};
```

Hasonlóan járunk el az opcionális jogosultságokkal.

```
export type PageRecommendedPermissions = {
  StationTasks: "Permission:Station:Read" | "Permission:Order:UpdateStatus";
  OrderHistory: never;
  AdminManager: "Permission:Admin:Create" | "Permission:Admin:Update" |
  "Permission:Admin:Delete";
  CustomerManager: "Permission:Customer:Update" | "Permission:Customer:Delete";
  // ...
};
```

Ezt követően hozzárendeljük a jogosultságokhoz az azokkal elérhető végpontokat, így biztosítva azt, hogy fejlesztő ne követhesse el azt a hibát, hogy helyesen dokumentált jogosultsággal rossz végpontot próbál meg elérni.

```
export type PermissionEndpoints = {
  "Permission:Admin:Create": { path: "/api/Admin"; method: "post" };
  "Permission:Admin:Delete": { path: "/api/Admin/{id}"; method: "delete" };
  "Permission:Admin:Read":
    | { path: "/api/Admin"; method: "get" }
    | { path: "/api/Admin/{id}"; method: "get" };
  "Permission:Admin:Update": { path: "/api/Admin/{id}"; method: "put" };
};
```



```
// ...
};
```

Ezután egy helper típust készítünk, melynek az a feladata, hogy az előbbi típus segítségével, jogosultságok alapján leszűkítse a http klienssel elérhető végpontokat.

```
type PickPaths<Paths extends paths, Filter extends { path: string; method: string }> = {
  [PathKey in Filter['path'] & keyof Paths]: Pick<
    Paths[PathKey],
    (Extract<Filter, { path: PathKey }>['method'] | 'parameters') & keyof Paths[PathKey]
  >;
};

export type HooksForPerm<Perm extends PermissionName> = ReturnType<
  typeof createClient<PickPaths<paths, PermissionEndpoints[Perm]>>
>;
```

A következő típus is ugyanígy jár el, azzal a különbséggel, hogy a jogosultságok nélkül elérhető (nyílt) végpontokra szűkíti a http klienst.

```
type NoPermEndpoints =
  | { path: "/api/User/me"; method: "get" }
  | { path: "/api/Shift/my"; method: "get" }
  | { path: "/api/Role/my"; method: "get" }
  | { path: "/api/Restaurant"; method: "get" }
  // ...

export type HooksWithNoPerm = ReturnType<
  typeof createClient<PickPaths<paths, NoPermEndpoints>>
>;
```

Ezután exportáljuk az implementálandó hook-ok típusát. 3 hook-ot és 1 metódust ad vissza a fő hook.

- `invalidateApiCache`: törli a gyorítótárazott kéréseket. Rendkívül hasznos kijelentkezéskor, mivel nem szeretnénk, ha egy másik felhasználó adatai maradjanak a cache-ben.
- `useNoPerm`: a jogosultság nélkül elérhető végpontokra szűkített http kliens-t adja vissza

Opcionálisan megadhatjuk az oldal nevét, ekkor elérhetővé válik a fentebbi két függvényen kívül további kettő.

- `useNecessaryPerm(permission)`: paraméterként kizárólag az oldalhoz nyilvántartott kötelező jogosultságot fogad el, és a jogosultsággal elérhető végpontokra korlátozott http klienst ad vissza:
- `useRecommendedPerm(permission)`: megegyezik az előzővel, csak a kötelező helyett az opcionális jogosultságokkal

```
export type UseApiWithoutSpecifiedPage = {
  useNoPerm(): HooksWithNoPerm;
  invalidateApiCache(): void;
};

type UseApiWithSpecifiedPage<Page extends PageName> = {
  useNecessaryPerm<Perm extends PageNecessaryPermissions[Page]>(
    permission: Perm,
  ): HooksForPerm<Perm>;
  useRecommendedPerm<Perm extends PageRecommendedPermissions[Page]>(
    permission: Perm,
  ): HooksForPerm<Perm> | null;
};

export type UseApi<Page extends PageName | null> = UseApiWithoutSpecifiedPage & (Page
  extends PageName ? UseApiWithSpecifiedPage<Page> : never);
```

Végül az `api.ts`-ben implementáljuk a fenti típust:

```
export function useApi<P extends PageName>(page: P): UseApi<P>;
export function useApi(page: null): UseApiWithoutSpecifiedPage;
export function useApi() {
```

```

const useNoPerm = (): HooksWithNoPerm => apiClient as unknown as HooksWithNoPerm;

const useHasPerm = (perm: PermissionName): boolean =>
  useNoPerm().useSuspenseQuery('get', '/api/Permission/my').data.includes(perm);

const useNecessaryPerm = <Perm extends PageNecessaryPermissions[NonNullable<PageName>]>(
  perm: Perm,
): HooksForPerm<Perm> => {
  if (!useHasPerm(perm)) {
    throw new Error('User does not have necessary permission: ' + perm);
  }
  return apiClient as unknown as HooksForPerm<Perm>;
};

const useRecommendedPerm = <Perm extends PageRecommendedPermissions[NonNullable<PageName>]>(
  perm: Perm,
): HooksForPerm<Perm> | null => {
  if (!useHasPerm(perm)) {
    return null;
  }
  return apiClient as unknown as HooksForPerm<Perm>;
};

return { useNoPerm, useNecessaryPerm, useRecommendedPerm, invalidateApiCache };
}

```

A fenti típusokat generáló kód a `generate-page-permission.ts` nevű fájlban található melyet node segítségével futtatunk egy `npm` paranccsal.

7.8 Middleware-ek

A middleware-ek az `api.ts`-ben találhatók és céljuk, hogy bizonyos események (pl. kérés, válasz, hiba) előtt/után lefussanak és módosítsák az eredményt a következő lépés végrehajtása előtt

7.8.1 AuthenticationMiddleware

Amennyiben amiatt történik egy hiba, mert a felhasználó nincs bejelentkezve, átirányítja a bejelentkező felületre.

7.8.2 ErrorResponseMiddleware

Amennyiben hiba történik tájékoztatja a felhasználót a sarokban megjelenő értesítés segítségével. A hiba okát a szerver által visszaküldött sztenderd `ProblemDetails` objektumból kérdezi le.



7.9 Pages mappa

Hogy könnyen meg tudjunk különböztetni egy sima komponenst egy oldaltól, minden oldalt ebben a mappában helyeztünk el, a router csak innen importálhat. A

7.10 Router

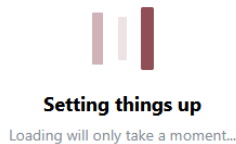
Itt adjuk meg, hogy melyik oldal milyen útvonalon legyen elérhető, valamint beállítjuk a fejléc címét és gombjait.

7.11 Hibakezelés

Amennyiben végzetes hiba történik a router az `ErrorPage` komponenst jeleníti meg. Így biztosítjuk, hogy hiba esetén se legyen túl negatív a felhasználói élmény.

7.12 Töltőképnyő

`Suspense` segítségével az API kérésekre való várakozás közben töltőképnyőt jelenítünk meg, így javítva a felhasználói élményt.



7.13 Backend url megadása

A `VITE_API_BASE_URL` környezeti változó segítségével megadhatjuk a szerver címét. Ez azért hasznos, mert fejlesztés közben még nem tudhatjuk, hogy az éles környezet milyen címen lesz elérhető. Erről bővebben a [Konfiguráció](#) fejezetben olvashatunk.

7.14 Összefoglaló

#	Cím	Útvonal	Publikus-e	Leírás
1	Welcome	/	igen	Kezdőoldal
2	Home	/home	nem	Főmenü
3	Login	/login	igen	Bejelentkező felület
4	Register	/register	igen	Regisztrációs felület
5	Place Order	/order	nem	Rendelésleadó felület
6	My Shifts	/my-shifts	nem	Alkalmazottak számára elérhető felület ahol megnézhetik a műszakjaikat
7	Order History	/order-history	nem	Rendelési előzmények megtekintésére szolgáló felület
8	Profile	/profile	nem	A felhasználó megtekintheti és szerkesztheti a saját profilját
9	About	/about	igen	Rövid leírás a projektről
10	Station Tasks	/station-tasks/:id?	nem	Megnézhetjük és frissíthetjük egy állomás feladatainak állapotát
11	Order Status Display	/order-status-display	nem	A lobbyban várakozó vendégek itt követhetik a rendelésük állapotát
12	Allergy Manager	/manage/allergy	nem	Itt lehet módosítani az ételallergiákat
13	Customer Manager	/manage/customer	nem	Itt lehet módosítani a vásárlók profiljait
14	Ingredient Manager	/manage/ingredient	nem	Itt lehet módosítani az összetevőket
15	Employee Manager	/manage/employee	nem	Itt lehet módosítani az alkalmazottak profiljait
16	Machine Manager	/manage/machine	nem	Itt lehet módosítani a gépek profiljait

17	Food Manager	/manage/food	nem	Itt lehet módosítani az ételeket
18	Menu Manager	/manage/menu	nem	Itt lehet módosítani a menüket
19	Order Manager	/manage/order	nem	Itt lehet módosítani a rendeléseket
20	Role Manager	/manage/role	nem	Itt lehet módosítani a szerepköröket
21	Shift Manager	/manage/shift	nem	Itt lehet módosítani az alkalmazottak műszakjait
22	Station Category Manager	/manage/station-category	nem	Itt lehet módosítani az állomások kategóriáit
23	Station Manager	/manage/station	nem	Itt lehet módosítani az állomásokat
24	Restaurant Manager	/manage/restaurant	nem	Itt lehet módosítani az éttermeket
25	Admin Manager	/manage/admin	nem	Itt lehet módosítani az adminok profiljait

8 Adatbázis

Adatbázisnak MariaDb-t választottunk. A relációkat kezdetben a DbSchema nevű szoftverrel modelleztük. Később az elkészített tervek alapján megírtuk az EF Core entitásokat és ezután a DbSchema-t már csak vizualizálásra és dokumentációra használtuk. **A szoftverrel készült mentést, a PDF és PNG formátumban exportált fájlt külön csatoltuk.**

8.1 ASP.NET Identity táblák

Ezek a táblák az Identity könyvtár által használt alapértelmezett táblák. A fő tábla az aspnetusers (felhasználók), ehhez kapcsolódóan tárolja a szerepköröket (aspnetroles) és a tokeneket (aspnetusertokens). Tároljuk továbbá a claim-eket, mind a szerepkörökhöz (aspnetroleclaims), mind a felhasználókhoz (aspnetuserclaims). A csak egyes típusú felhasználókhoz tárolt adatok külön táblákban vannak: admins, employees, machines, customers. Ez egyben jelzi is, hogy az adott felhasználó milyen típusú (a C# entitások öröklődése Table Per Type [TPT] módszerrel vannak tárolva).

8.2 Étlaphoz kapcsolódó táblák

Itt 4 fő entitás van: allergies, ingredients, foods, menus. Ezek között many-to-many kapcsolat van, ezeket kapcsolótáblákkal reprezentáljuk. A menu és food, valamint a food és ingredient közötti kapcsolótábla extra információt is tárol (előbbinél quantity [a menüben szereplő ételek mennyisége], utóbbinál a quantity és unit [milyen a mértékegységben értjük a mennyiséget, pl. liter, gramm]).

8.3 Rendelésekhez kapcsolódó táblák

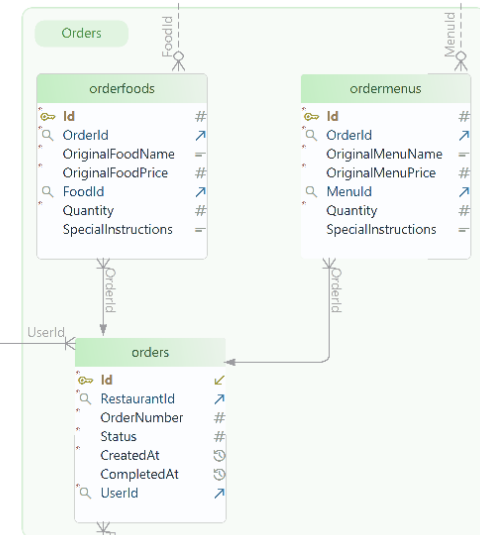
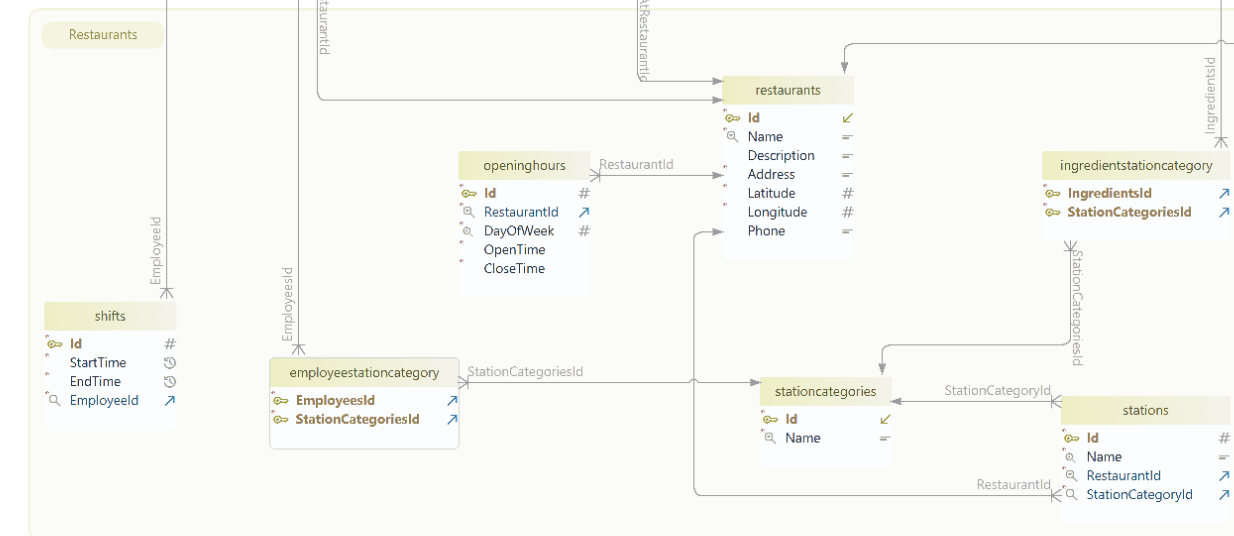
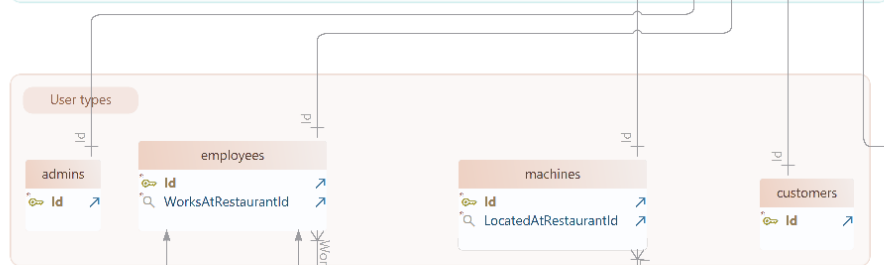
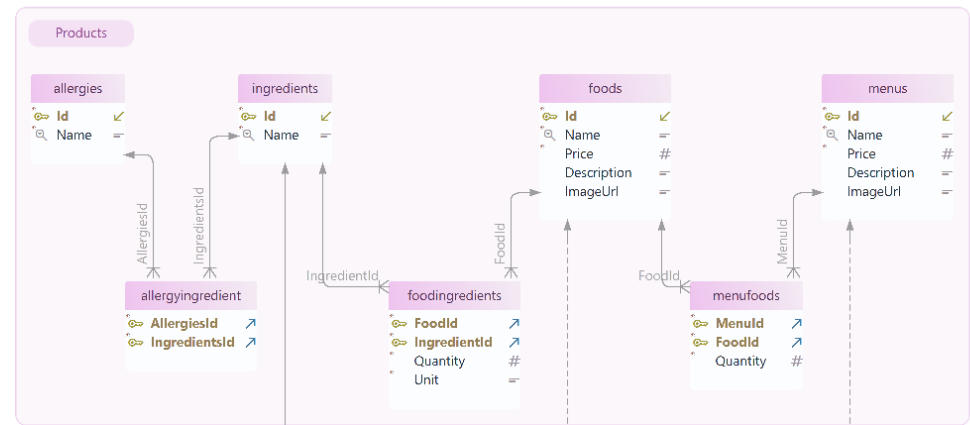
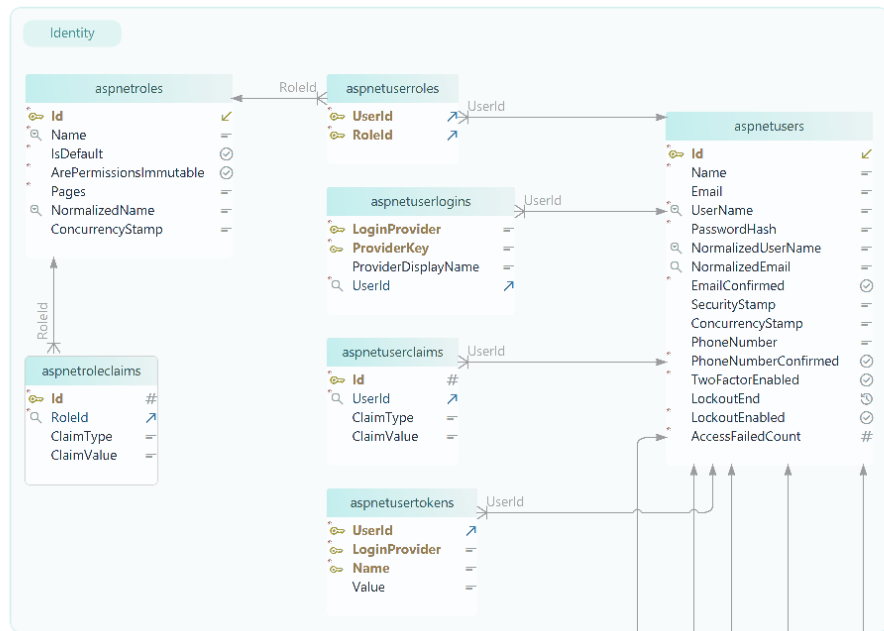
A rendeléseket az orders táblában tároljuk. Szerepel benne az étterem, ahol leadták a rendelést, a felhasználó, aki leadta a rendelést, a rendelés státusza, illetve teljesítés ideje. Ezen kívül rögzítésre kerül a termékek rendelés pillanatában nyilvántartott neve és ára, így **biztosítva azt, hogy a rendelési előzmények a termék módosítása vagy törlése után is pontosak maradjanak**. A rendelés állhat menükből és önálló ételekből, ezeket külön táblákban tároljuk, ahol nyilván van tartva a rendelt mennyiség és a speciális kérések.

8.4 Éttermekhez kapcsolódó táblák

Az éttermeket a restaurants táblában tároljuk, itt szerepel az étterem neve, leírása, címe, koordinátája és telefonszáma. Az openinghours tábla kapcsolódik az étteremhez, itt tároljuk a nyitvatartást: szerepel benne a hét napja, nyitás és zárás. A hét napjára és az étteremre egyedi megkötést (unique constraint) tettünk, így biztosítva, hogy minden napra csak egy időintervallumot tárolunk.

A stationcategory táblában tároljuk a lehetséges állomástípusokat (pl. burgonya állomás, grillező állomás, burger állomás) és a stations táblában pedig a konkrét állomásokat. A konkrét állomásokhoz eltároljuk, hogy melyik étteremben van elhelyezve. A employestationcategory kapcsolótáblával reprezentáljuk azt, hogy melyik alkalmazott milyen típusú állomásokra van betanítva.

A shift tábla tárolja a műszakokat. Ebben tároljuk a műszak kezdetét és végét, valamint az alkalmazott azonosítóját.



9 API dokumentáció

Az API a REST architektúrát követi, az adatok átvitele pedig JSON formátumban történik. A legtöbb helyen igyekeztünk elkerülni a mélyen beágyazott (nested) adatok küldését, mert ez könnyebbé tette a backend fejlesztését. Ez alól kivétel néhány végpont, ahol viszonylag nagy mennyiségű adat esik át komplex feldolgozáson (pl. /api/station/:id/tasks). **A teljes API dokumentációt az OpenAPI sémából generáltuk és külön dokumentumként csatoltuk.** A pdf generáláshoz a nyílt forráskódú [RapiPDF](#) eszközt használtuk. A példa kedvéért azonban álljon itt néhány végpont magyar nyelvű dokumentációja

9.1 Példák

9.1.1 GET /api/Allergy

Lekéri az összes rögzített allergént a rendszerből. A művelethez Permission:Allergy:Read jogosultság szükséges.

Kérés: Nem igényel paramétereket.

Válaszok:

- 200 OK - Egy objektumokból álló tömb:
 - name (string): Az allergén megjelenített neve.
 - ingredientIds (array): A kapcsolódó összetevők azonosítói.
 - id (integer): Egyedi azonosító.
- 400 Bad Request - Hibás kérés esetén visszaadott ProblemDetails objektum:
 - type (string): A hiba típusa.
 - title (string): A hiba rövid címe.
 - status (integer): A HTTP státuszkód.
 - detail (string): A hiba részletes leírása.
 - instance (string): A hibát kiváltó kérés azonosítója.
- 401 Unauthorized - A kérés hitelesítés nélkül érkezett, vagy az azonosítás sikertelen volt.
- 403 Forbidden - A hitelesített felhasználónak nincs jogosultsága a művelet elvégzéséhez.

9.1.2 POST /api/Auth/login

Felhasználó hitelesítése felhasználónév és jelszó segítségével. Sikeres azonosítás esetén hozzáférési tokenet ad vissza.

Query paraméterek:

- useCookies (boolean): Mentse-e a tokenet sütibe.
- useSessionCookies (boolean): A sütik törlődjenek-e a böngésző bezárása után.

Kérés törzse (application/json):

- userName (string): Felhasználónév.
- password (string): Jelszó.

Válaszok:

- 200 OK - Bearer hitelesítés esetén hozzáférési token válasz, süti alapú hitelesítés esetén üres válasz:
 - tokenType (string): A token típusa, amennyiben nem sütiben kértük.
 - accessToken (string): A hozzáférési token, amennyiben nem sütiben kértük.
 - expiresIn (integer): Lejáratási idő másodpercben, amennyiben nem sütiben kértük.

- refreshToken (string): A frissítési token, amellyel új hozzáférési token igényelhető anélkül, hogy újra be kellene jelentkezni.
- 401 Unauthorized - A megadott felhasználónév, illetve jelszó érvénytelen.

9.1.3 GET /api/User/me

Lekéri a jelenleg bejelentkezett felhasználó profiladatait.

Kérés: Nem igényel paramétereket.

Válaszok:

- 200 OK - A bejelentkezett felhasználó profilja:
 - name (string): A felhasználó megjelenített neve.
 - userName (string): A felhasználó bejelentkezési neve.
 - roleIds (array): A felhasználóhoz rendelt szerepkörök azonosítói.
 - userType (string): A felhasználó típusa.
 - orderIds (array): A felhasználóhoz tartozó rendelések azonosítói.
 - id (integer): A felhasználó egyedi azonosítója.
- 401 Unauthorized - A kérés hitelesítés nélkül érkezett, vagy az azonosítás sikertelen volt.
- 404 Not Found - A bejelentkezett felhasználóhoz nem található profiladat a rendszerben.

9.2 Hitelesítés (Authentication)

A rendszer Bearer HTTP hitelesítést használ JWT (JSON Web Token) alapokon. A kliens a /api/Auth/login végpontról kérhet tokenet és a választ süti vagy JSON formában várhatja. JSON esetén a kliensnek a tokenet a kérések fejlécében kell küldenie, süti esetében ez automatikus.

9.3 Hibakezelés

A FastFuel API egységes hibaobjektumot ad vissza hiba esetén (pl. 400-as kódnál), amely a következő mezőket tartalmazza:

- type (string): A hiba típusa.
- title (string): A hiba rövid címe.
- status (integer): HTTP állapotkód.
- detail (string): Részletes leírás a hiba okáról.
- instance (string): A hiba előfordulásának helye (URI).

Gyakori hibakódok:

- 401 Unauthorized: Nincs érvényes hitelesítés.
- 403 Forbidden: A felhasználónak nincs jogosultsága a művelethez.
- 404 Not Found: A kért erőforrás nem található.
- 409 Conflict: Ütközés (pl. már létező felhasználónév).

10 Üzembehelyezési útmutató

10.1 Éles környezet

10.1.1 Előfeltételek

Production környezetben való használathoz szükséges a Docker telepítése.

10.1.2 Konfiguráció

Production környezetben a **docker-compose.yml** fájlban van lehetőség konfigurációra. Mindegyik service-n belül található egy **environment** nevű lista, ebben találhatók az adott szolgáltatás környezeti változói. Ezek a következők:

- db (adatbázis):
 - MARIADB_ROOT_PASSWORD: root jelszó az adatbázishoz
 - MARIADB_DATABASE: adatbázis neve
 - MARIADB_USER: felhasználónév az adatbázishoz
 - MARIADB_PASSWORD: jelszó az adatbázishoz
- server:
 - ASPNETCORE_ENVIRONMENT: jelzi, hogy a környezet Production (alapértelmezett) vagy Development
 - ASPNETCORE_URLS: Ezen a címen fog a szerver hallgatózni
 - ConnectionStrings__DefaultConnection: Sztenderd connection string, itt próbálja majd a szerver elérni az adatbázist
- client:
 - VITE_API_BASE_URL: Ezen a címen fogja a webkliens elérni a szervert

Minden service-nek jelenleg expose-olva van a portja a **ports** változó segítségével, amin keresztül kívülről elérhetőek. Ha szeretnénk valami webcímen, esetleg https kapcsolattal elérhetővé tenni, akkor szükség lesz egy reverse proxy-ra is. Mi ehhez Traefik-et használunk.

Az alapértelmezett admin felhasználó belépési adatai:

Felhasználónév: admin

Jelszó: Admin123!

10.1.3 Beüzemelés

A **cd** parancs segítségével navigáljunk a kliens gyökérmappájába. Telepítsük a szükséges csomagokat az **npm i** parancs segítségével.

A **cd** parancs segítségével navigáljunk a projekt gyökérmappájába. Ezt követően az indításhoz adjuk ki a **docker compose up -d** parancsot. Linux környezetben ehhez gyakran szükség lehet adminisztrátori (sudo) jogosultságokhoz.

10.2 Demo környezet

Beüzemelése megegyezik a Production környezettel, azzal a különbséggel, hogy az ASPNETCORE_ENVIRONMENT változót „Demo”-ra kell állítani. Ekkor egy production build fog elindulni, de létrejönnek a fejlesztési környezetből ismert tesztadatok. Ilyenkor a következő felhasználók állnak rendelkezésre:

- admin – Admin123!
- customer – Customer123!
- employee – Employee123!
- machine – Machine123!
- shiftmanager – ShiftManager123!

Mi is üzembe helyeztük egy Demo példányt a <https://fastfuel.cs-z.hu/> címen. Ez egy ingyenes Oracle Cloud szerveren, Docker környezetben fut és Traefik segítségével kommunikál a külvilággal. A domain Cloudflare segítségével van kezelve (proxy nélkül). Https kapcsolatot is biztosítunk automatikusan megújuló Let's Encrypt tanúsítvánnyal.

10.3 Fejlesztési környezet

10.3.1 Előfeltételek

10.3.1.1 Backend

.NET10 telepítése, azzal kompatibilis IDE telepítése, Nuget csomagok telepítése, MariaDb telepítése.

10.3.1.2 Frontend

Node telepítése, függőségek telepítése az `npm install` parancs segítségével

10.3.2 Környezeti változók

Az éles környezet [Konfiguráció](#) fejezetében tárolt változókat állítsuk be a backend projektben például a .NET User Secrets segítségével. Ezután indítsuk el. Ezt követően a frontendet is elindíthatjuk az `npm run dev` parancs segítségével.

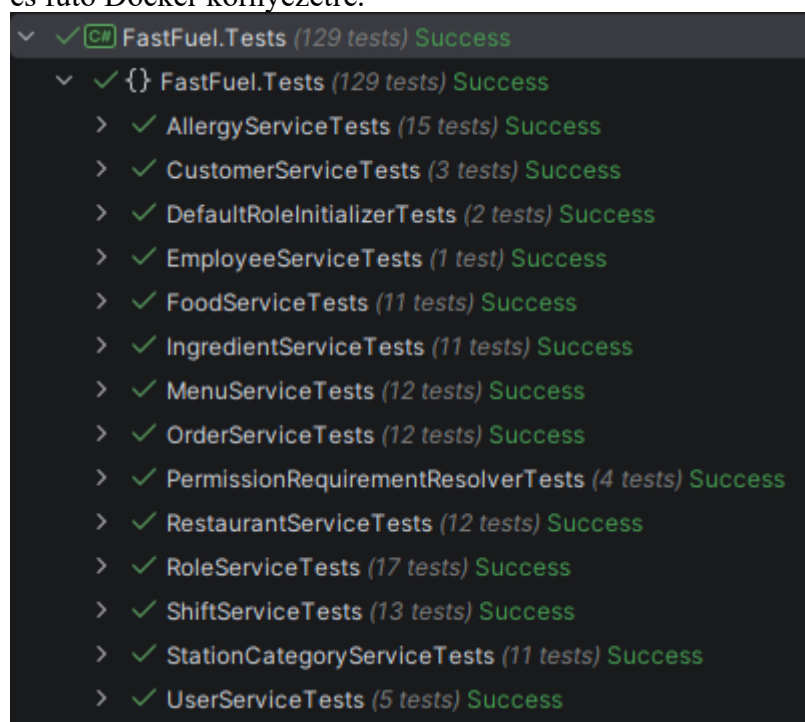
11 Tesztelés

11.1 Áttekintése

A projekt készítése során automata és manuális tesztek végeztünk. Előbbi kategóriába tartoznak az egységtesztek (unit tests), utóbbiba az API és az end-to-end (E2E) tesztek. Ugyan 100% lefedettség szinte lehetetlen, mi törekedtünk rá.

11.2 Egységtesztek

Az egységtesztek a FastFuel.Tests nevű projektben találhatók meg, a backend solution-ban. Elkészítésük során nagy kihívást jelentett az adatbázis mock-olása. Erre jó megoldás lett volna a Repository Pattern, de mi a TestContainers nevű könyvtár mellett döntöttünk egyszerűsége és a fejlesztés gyorsasága miatt. Az egységtesztek végrehajtásához szükségünk lesz telepített és futó Docker környezetre.



11.3 API tesztek

Az API teszteléshez Postmanbe importáltuk az OpenAPI sémát, ez nagyban megkönnyítette a manuális tesztelést.

A Postman collection-t a docs mappában találjuk, JSON formátumban exportálva. Kezdsnek szükséges megadni a baseUrl-t, ami development környezetben <http://localhost:5249>. Ezután be kell jelentkezni és a kapott tokent bemásolni a *bearerToken* változóba.

11.4 E2E manuális tesztek

A manuális E2E tesztek terjedelme miatt külön dokumentumként csatoljuk a projekthez.

12 Jövőbeli fejlődési lehetőségek

12.1 Funkcionális lehetőségek

- Hűségprogram és CRM modul: Egy pontgyűjtő rendszer integrálása, amely lehetővé teszi a törzsvásárlók azonosítását és személyre szabott kedvezmények nyújtását.
- Készletgazdálkodás és automatikus rendelés: A rendelés kezelés össze lenne kapcsolva a készletnyilvántartással. A rendszer az eladott ételek alapján valós időben követné le az alapanyag mennyiségét, és jelezné (vagy automatikusan megrendelné), ha valami fogyóban van.
- Külső házhozszállítási platformok integrációja: API kapcsolat kiépítése olyan szolgáltatókkal, mint a Wolt vagy a Foodora, hogy a külső rendelések is közvetlenül a lánc konyhai kijelzőire érkezzenek.

12.2 Nem funkcionális lehetőségek

- Repository pattern implementálása a könnyebb tesztelhetőség érdekében
- E2E tesztelés automatizálása Cypress segítségével
- A modern trendeket követő UI design

13 Felhasznált források

- [W3Schools](#)
- [Mantine dokumentáció](#)
- [Tabler ikonok](#)
- [Vite dokumentáció](#)
- [Milan Jovanović YouTube csatornája](#)
- [ASP.NET dokumentáció](#)
- [StackOverflow](#)
- [OpenVerse Creative Commons képek](#)
- [RapiPDF](#)